

第2章 CPU管理

- 2.1 进程的概念
- 2.2 多进程图像
- 2.3 CPU调度策略
- 2.4 进程控制
- 2.5 用户级线程
- 2.6 内核级线程

单选题 1分

操作系统在计算机系统中位于____之间。

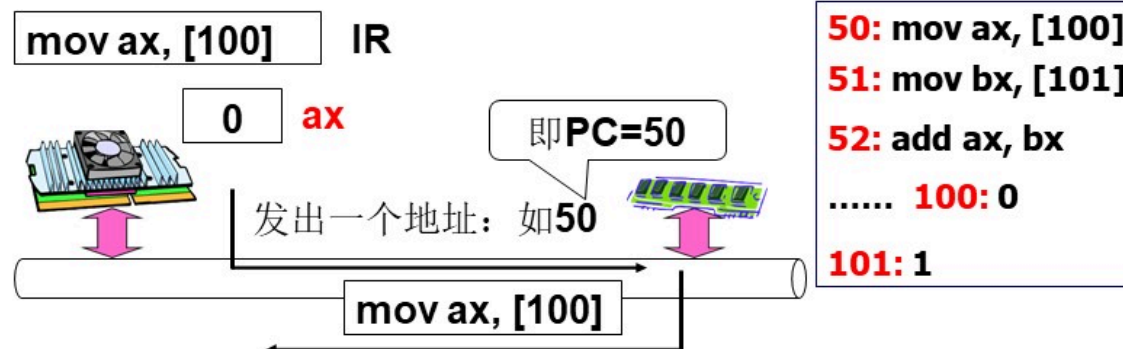
- ☐ A CPU和用户
- ☐ B CPU和内存
- ☐ C 计算机硬件和用户
- ☐ D 计算机硬件和软件

2.1 进程的概念



管理CPU，先要使用CPU...

■ CPU的工作原理



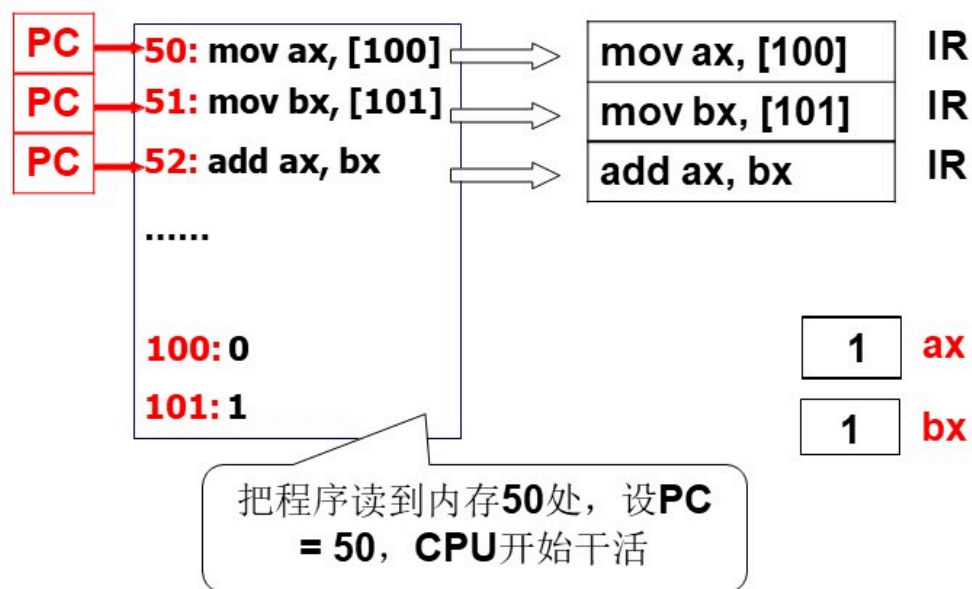
■ CPU怎么工作?

■ CPU怎么管理?

CPU上电以后发生了什么?
自动的取指—执行

管理CPU的最直观方法

■ 设好PC初值就完事!



看看这样做有没有问题？

```
int main(int argc, char*
argv[])
{
int i, to, *fp, sum = 0; to =
atoi(argv[1]); for(i=1;
i<=to; i++)
{
sum = sum + i;
fprintf(fp, "%d", sum);
}
}
```



fprintf (IO指令) 用一条
其他计算语句代替

```
C:\>sum 10000000
0.015000 seconds
```

$0.015/10^7$

有fprintf

```
C:\>sum 1000
0.859000 seconds
```

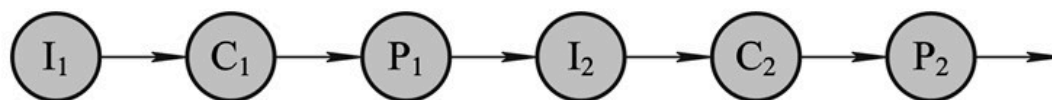
$0.859/10^3$

$5.7 \times 10^5 : 1$

看看这样做有没有问题？

程序的顺序执行：

一个具有独立功能的程序独占CPU运行，直到获得最终结果的过程。



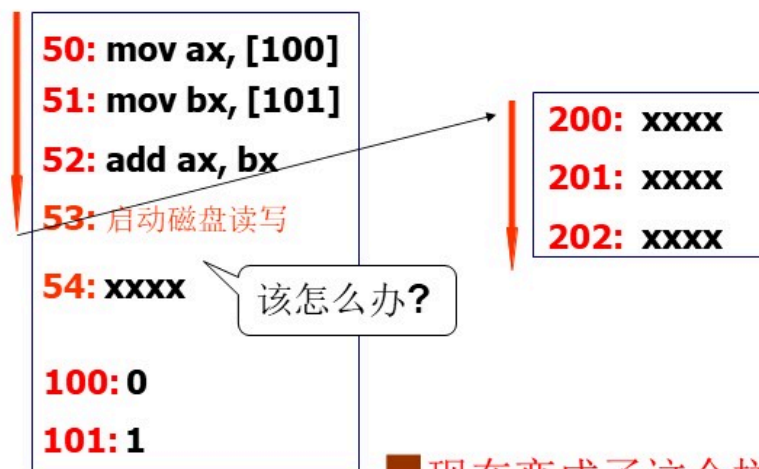
特点：

（1）顺序性；（2）封闭性；（3）可再现性。

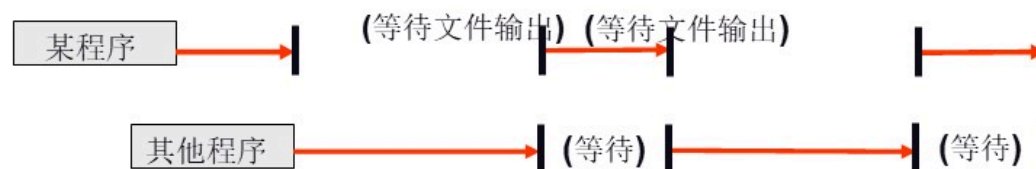
优点：程序的编制、调试方便；

缺点：CPU与外部设备之间不能并行工作，资源利用率低，计算机系统效率不高。

怎么解决?

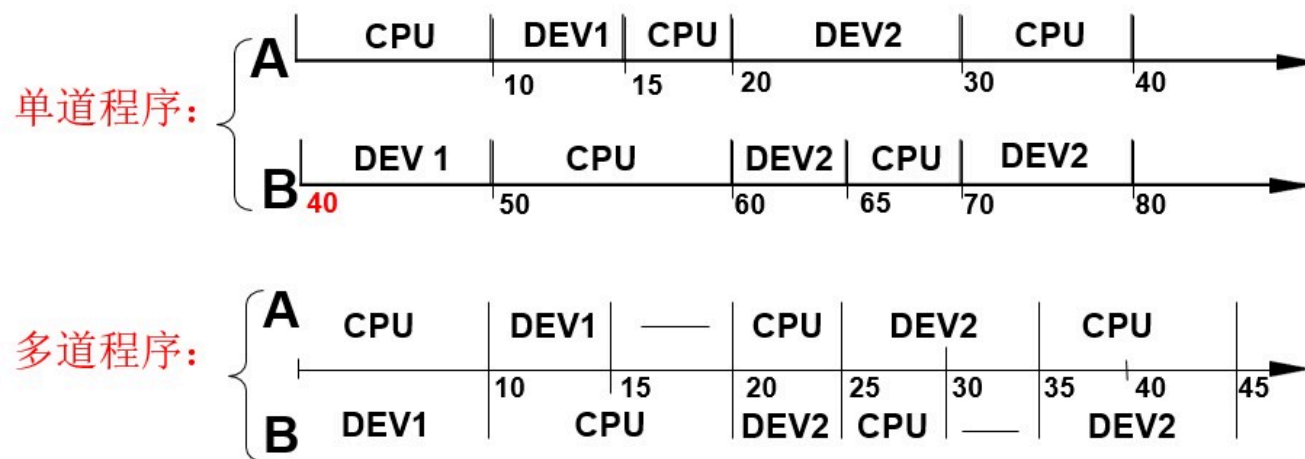


■ 现在变成了这个样子

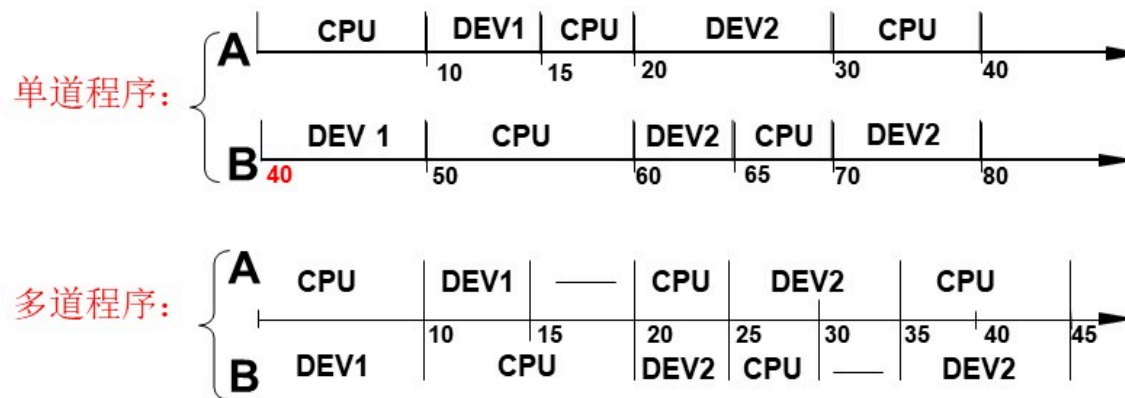


填空题 6分

单道程序：CPU利用率 [填空1]；DEV1利用率 [填空2]；DEV2利用率 [填空3]。
 多道程序：CPU利用率 [填空4]；DEV1利用率 [填空5]；DEV2利用率 [填空6]。



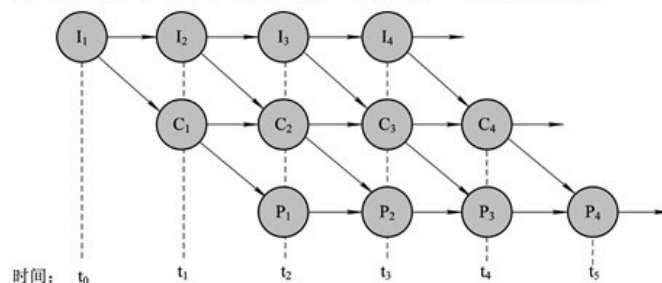
多道程序、交替执行



	单道程序	多道程序
CPU利用率	$40/80=50\%$	$40/45=89\%$
DEV1利用率	$15/80=18.75\%$	$15/45=33\%$
DEV2利用率	$25/80=31.25\%$	$25/45=56\%$

多道程序、交替执行

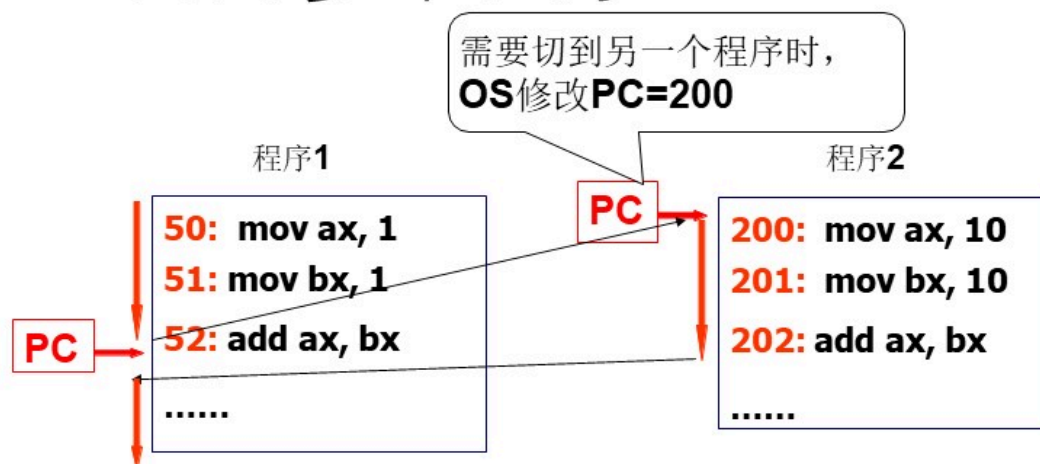
- 程序的并发执行：
- 多个程序在执行时间上是重叠的。



注意：在多道程序设计环境下，多个程序可以在单CPU上交替执行，也可以在多个CPU上并行执行。程序的并发执行通常是指多个程序在单个CPU上的交替执行。

多道程序的并发执行是指多个程序在宏观上的并行，微观上的串行。

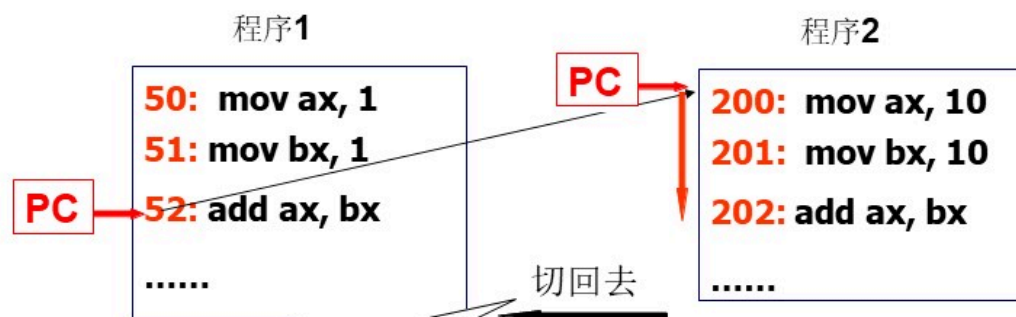
一个CPU面对多个程序？



■ 一个**CPU**上交替的执行多个程序：并发

■ 怎么做到？

修改寄存器PC就行了吗？



运行的程序和静态
程序不一样了...

怎么办？

程序1信息

2	ax
1	bx
53	PC

■ 要记录返回地址，要记录ax...

每个程序有了一个存放信息的
结构: **PCB**

引入“进程”概念



■ 运行的程序和静态程序不一样!

■ 需要描述这些不一样...

■ 程序 + 所有这些不一样 → 一个概念

这些不一样就成了
进程概念的外延

所有的不一样都表
现在**PCB**中...

■ 进程是进行（执行）中的程序

■ 进程有开始、有结束，程序没有

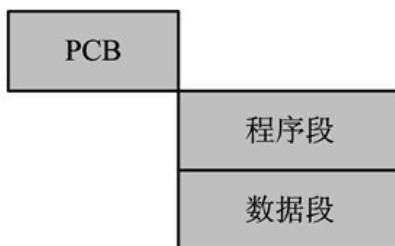
■ 进程会走走停停，走停对程序无意义

■ 进程需要记录**ax, bx, ...**，程序不用

多道进程，交替执行

2.1 进程的概念

- **进程**: 是一个可并发执行的具有独立功能的程序关于某个数据集合的一次执行过程, 也是操作系统进行资源分配和保护的基本单位。
- **进程的结构**
- PCB--用来存储程序向前推进的执行过程中所要记录的有关运行信息。
- 进程实体 = 程序段 + 相关数据段 + PCB



2.1 进程的概念

- 进程的特征:

- (1) 动态性; (2) 并发性; (3) 独立性;
- (4) 异步性; (5) 结构性。

- 进程与程序的区别:

- (1) 程序是指令的有序集合, 是一个静态的概念, 其本身没有任何运行的含义; 进程是程序在CPU上的一次执行过程, 是一个动态的概念。
- (2) 程序作为软件资料可以长期保存, 而进程则有生命期, 它因创建而诞生、因调度而执行、因得不到资源而暂停执行、因撤销而消亡。
- (3) 进程是一个独立运行的基本单位, 也是系统进行资源分配和调度的基本单位; 而程序作为静态文本既不运行, 也不分配和调度。
- (4) 进程与程序之间无一一对应关系。既然进程是程序的一次执行, 那么一个程序的多次执行可以产生多个进程, 而不同的进程也可以包含同一个程序。
- (5) 程序是记录在介质 (如磁盘) 上指令的有序集合, 而进程则由程序段、数据段和PCB这三部分组成。

单选题 1分

以下对进程的描述中，错误的是

- ☐ A 进程是动态的概念
- ☐ B 进程执行需要CPU
- ☐ C 进程是有生命期的
- ☐ D 进程是指令的集合

单选题 1分

以下关于进程的描述中，正确的是

- ☐ A 进程获得CPU运行是通过调度得到的
- ☐ B 优先级是进程调度的重要依据，一旦确定就不能改变
- ☐ C 在单CPU系统中，任何时刻都有一个进程处于运行状态
- ☐ D 进程申请CPU得不到满足时，其状态变为阻塞

2.2 多进程图像

多个进程使用CPU的图像

■ 如何使用CPU呢？

让程序执行起来

■ 如何充分利用CPU呢？

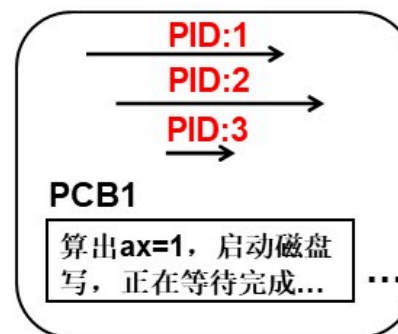
启动多个程序，交替执行

■ 启动了的程序就是进程，所以是多个进程推进

操作系统只需要把这些进程记录好、要按照合理的次序推进

(分配资源、进行调度)

这就是多进程图像...



多进程图像从启动开始到关机结束

- **main**中的**fork()**创建了第**1**个进程

init执行了**shell**(**Windows**桌面)

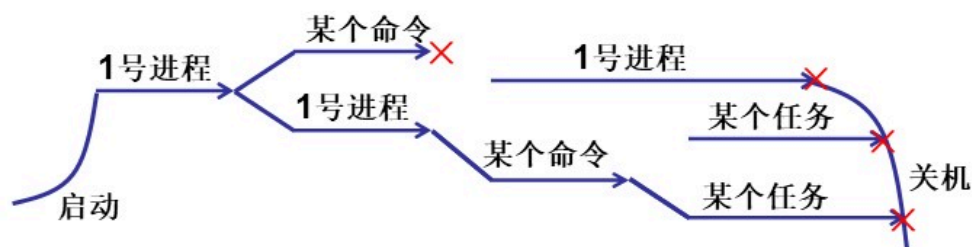
```
if(!fork()){init();}
```

- **shell**再启动其他进程

```
int main(int argc, char * argv[])  
{ while(1) { scanf( "%s" ,cmd);  
  if(!fork()){exec(cmd);} wait(); } }
```

一命令启动一个进程，返回**shell**再启动其他进程...

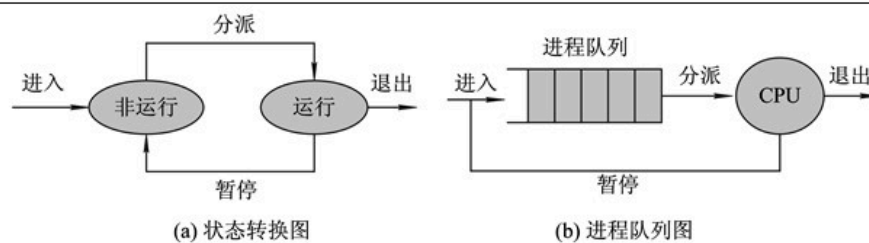
```
login as: tnp  
tq0192.168.1.1's password:  
Last login: Mon Nov 13 23:47:50 2006 from 192.168.1.2  
[tq0192null ~]$ ls  
work.txt  
[tq0192null ~]$
```



多进程图像：多进程如何组织？

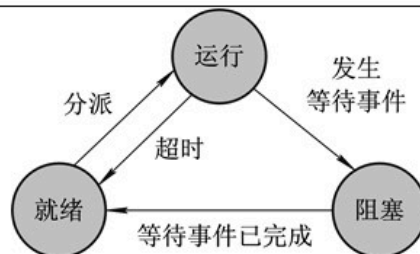
- 两状态进程模型

- 两种状态：运行 (Running) 和非运行 (Not-running)



三态模型

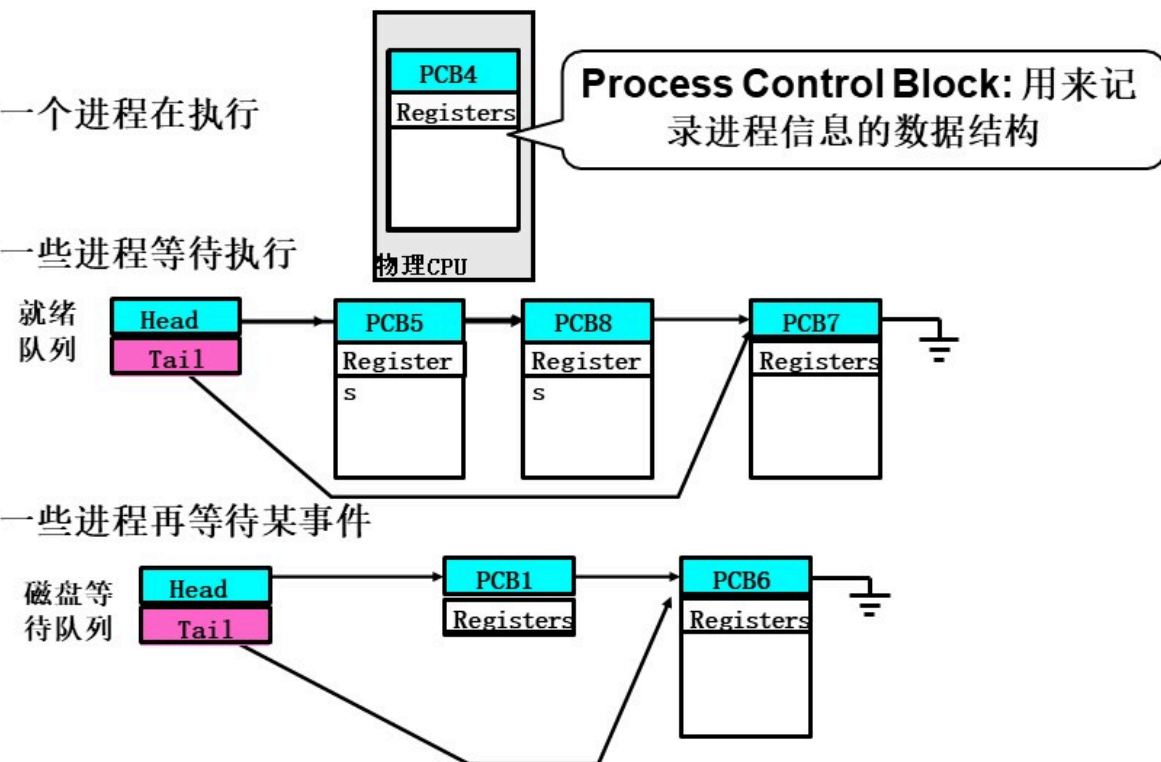
三种基本状态：运行、阻塞和就绪



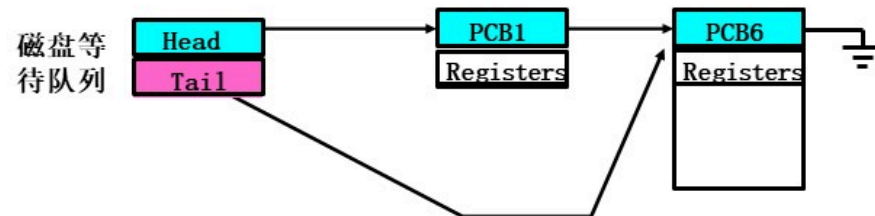
多进程图像：多进程如何组织？

- 有一个进程在执行

- 有一些进程等待执行



- 有一些进程再等待某事件

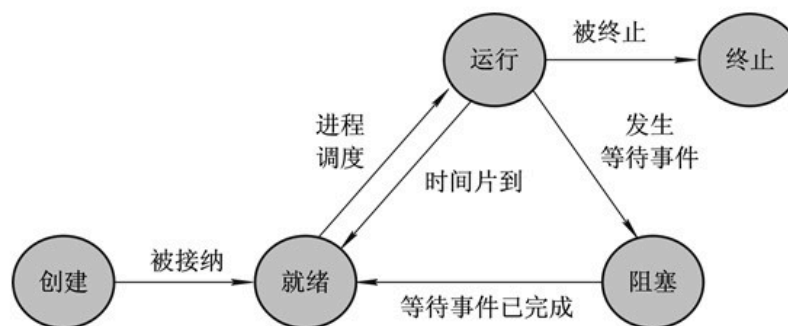


多进程的组织：PCB+状态+队列

■ 运行→等待；运行→就绪；就绪→运行.....

- 五态模型

- 五种状态：创建、就绪、运行、阻塞、终止



该图称为**进程状态图**

它能给出进程生存期的清晰描述

它是认识操作系统进程管理的一个窗口

多进程的组织：PCB+状态+队列

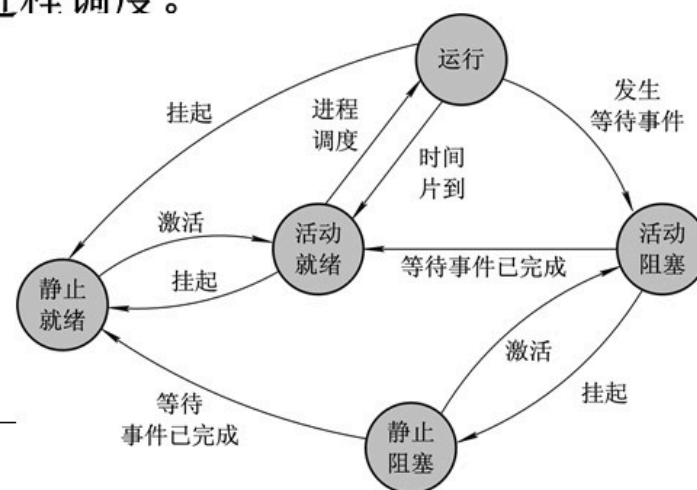
进程的挂起

指在内存中的进程被暂时移到外存（如磁盘）的过程。

当某个进程被挂起时，若被挂起的进程处于运行状态则停止执行；若被挂起的进程处于就绪状态则暂时不参加进程调度。

引起进程挂起的原因：

- (1) 用户的请求；
- (2) 父进程的请求；
- (3) 操作系统的原因。



多进程的组织：PCB+状态+队列

- 进程控制块

- PCB包含以下信息：
 - (1) 进程标识符。
 - (2) 进程的当前状态。
 - (3) 进程相应的程序段和数据段地址。
 - (4) 进程资源清单。
 - (5) 进程优先级。
 - (6) CPU现场保护区。
 - (7) 进程同步和通信机制。
 - (8) PCB队列指针或链接字。
 - (9) 与进程相关的其他信息。

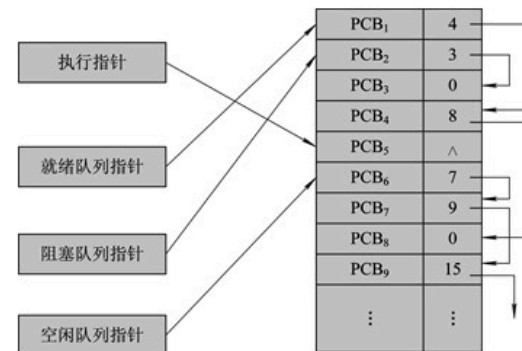
多进程的组织：PCB+状态+队列

PCB组织方式

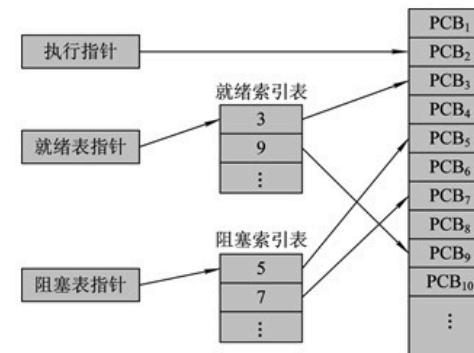
(1) 线性表方式



(2) 链接表方式



(3) 索引表方式



多进程图像：多进程如何交替？

启动磁盘读写；

pCur.state= 'W' ;

将**pCur**放到**DiskWaitQueue**;

schedule();

schedule()

{

pNew = getNext(ReadyQueue);

switch_to(pCur,pNew);

}

交替的三个部分：队列操作+调度+切换

- 就是进程调度，一个很深刻的话题

- **FIFO?**

- **FIFO**显然是公平的策略
- **FIFO**显然没有考虑进程执行的任务的区别

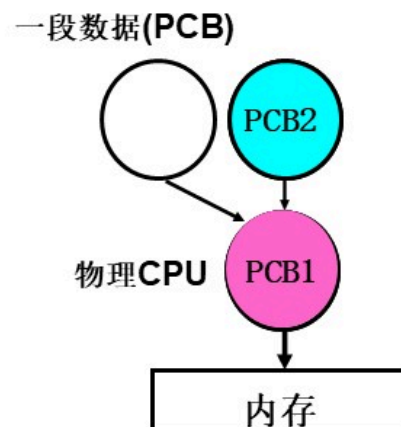
- **Priority?**

- 优先级该怎么设定？可能会使某些进程饥饿

交替的三个部分：队列操作+调度+切换

```
switch_to(pCur, pNew)
{
    pCur.ax = CPU.ax;
    pCur.bx = CPU.bx;
    ...
    pCur.cs = CPU.cs;
    pCur.retpc = CPU.pc;

    CPU.ax = pNew.ax;
    CPU.bx = pNew.bx;
    ...
    CPU.cs = pNew.cs;
    CPU.retpc = pNew.pc; }
}
```



多进程图像：多进程如何影响？

■ 多个进程同时存在于内存会出现下面的问题

进程1代码

```
mov ax, 10100b  
mov [100], ax  
.....
```

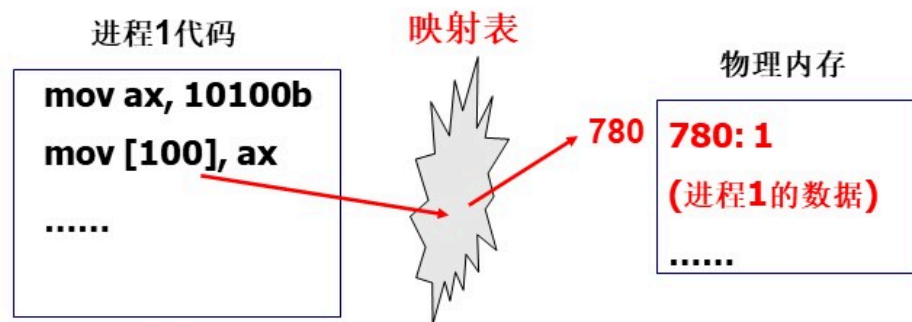
进程2代码

```
100: 00101  
.....
```

■ 解决的办法：限制对地址**100**的读写

■ 多进程的地址空间分离：内存管理的主要内容

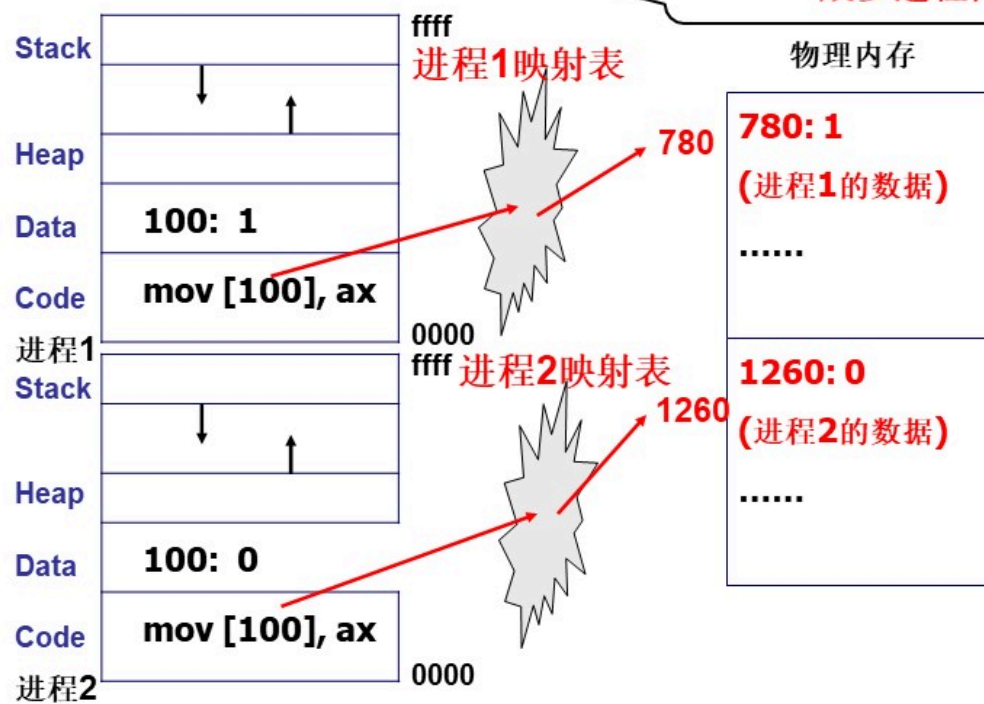
进程执行时的100...



- 进程1的映射表将访问限制在进程1范围内
- 进程1根本访问不到其他进程的内容
- 内存管理...

进程带动内存的使用

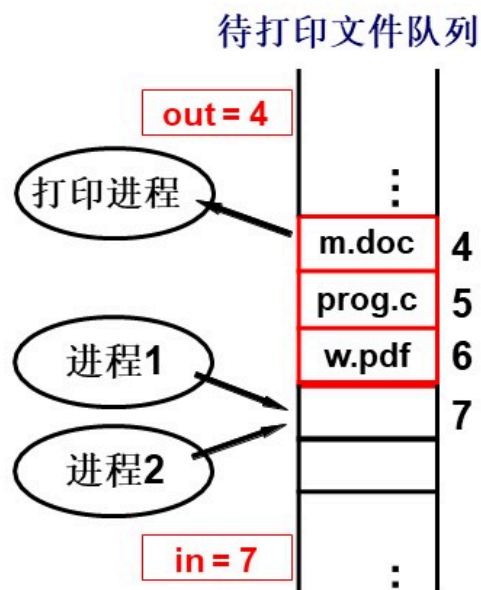
为什么说进程管理连带内存管理形成多进程图像？



多进程图像：多进程如何合作？

■ 想一想打印工作过程

- 应用程序提交打印任务
- 打印任务被放进打印队列
- 打印进程从队列中取出任务
- 打印进程控制打印机打印



从纸上到实际：生产者消费者实例

生产者进程

```
while (true) {  
    while(counter == BUFFER_SIZE)  
        ;  
    buffer[in] = item;  
    in = (in + 1) % BUFFER_SIZE;  
    counter++;  
}
```

消费者进程

```
while (true) {  
    while(counter == 0)  
        ;  
    item = buffer[out];  
    out = (out + 1) % BUFFER_SIZE;  
    counter--;  
}
```

共享数据

```
#define BUFFER_SIZE 10  
typedef struct { ... } item;  
item buffer[BUFFER_SIZE];  
int in = out = counter = 0;
```

两个合作的进程都要修改counter

一个可能的执行序列

初始情况

```
int counter=0;
```

生产者进程

```
counter++;
```

消费者进程

```
counter--;
```

```
P.register = counter;  
P.register = P.register + 1;  
C.register = counter;  
C.register = C.register - 1;  
counter = P.register;  
counter = C.register;
```

共享数据

```
counter = 5;
```

生产者P

```
register = counter;  
register = register + 1;  
counter = register;
```

消费者C

```
register = counter;  
register = register - 1;  
counter = register;
```

核心在于进程同步(合理的推进顺序)

■ 写 **counter** 时阻断其 他
进程访问 **counter**

一个可能的执行序列

```
P.register = counter;  
P.register = P.register + 1;  
C.register = counter;  
C.register = C.register - 1;  
counter = P.register;  
counter = C.register;
```

生产者P

给 **counter** 上锁

```
P.register = counter;  
P.register = P.register + 1;
```

消费者C

检查 **counter** 锁

生产者P

```
counter = P.register;
```

给 **counter** 开锁

消费者C

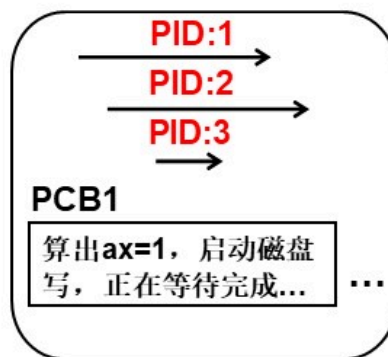
给 **counter** 上锁

```
C.register = counter;  
C.register = C.register - 1;  
counter = C.register;
```

给 **counter** 开锁

总结

如何形成多进程图像？



- 读写**PCB**, **OS**中最重要的结构, 贯穿始终
- 要操作寄存器完成切换
- 要写调度程序
- 要有进程同步与合作
- 要有地址映射

单选题 1分

在单CPU系统中实现并发技术后，_____。

- A** 各进程在某一个时刻并发运行，CPU 与I/O 设备间并行工作
- B** 各进程在一个时间段内并发运行，CPU 与I/O 设备间串行工作
- C** 各进程在一个时间段内并发运行，CPU 与I/O 设备间并行工作
- D** 各进程在某一个时刻并行运行，CPU 与I/O 设备间串行工作

单选题 1分

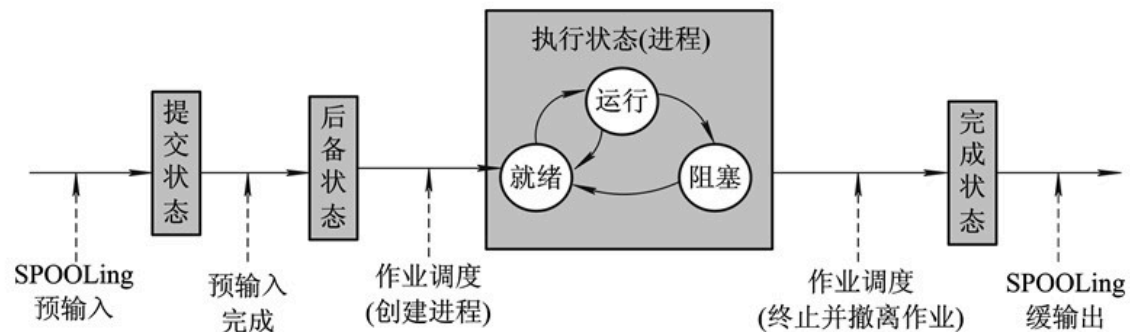
【考研真题】当一个进程处于正等着_____状态时，称为阻塞状态。

- A 等待协作进程的一个消息
- B CPU调度
- C 分给它一个时间片
- D 进入内存

2.3 CPU调度策略

作业与进程的关系

1. 批处理作业与进程的关系



作业状态4种:

(1) 提交状态; (2) 后备状态; (3) 执行状态; (4) 完成状态。

2. 分时系统中作业与进程的关系

2.3 CPU调度策略

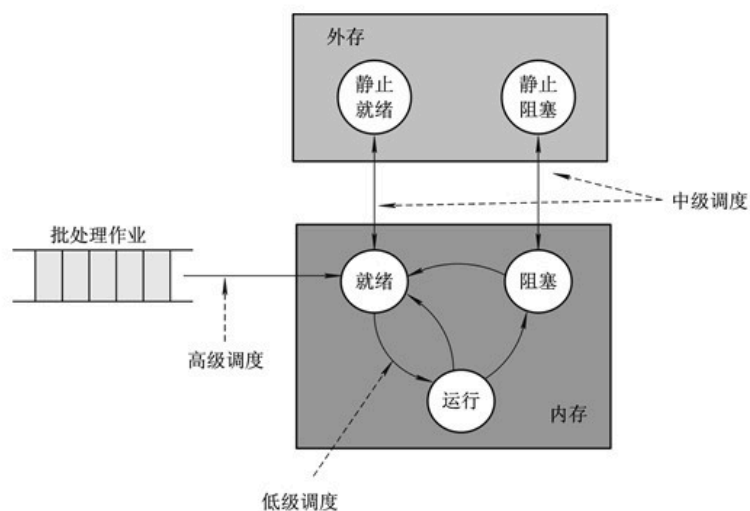
CPU的三级调度

1. 高级调度（校长）

又称作业调度或宏观调度，功能是按一定的调度算法把外存上处于后备作业队列中的作业调入内存，为它们分配所需的资源并创建进程，然后将新创建的进程插入到系统的就绪队列中。

功能：

- (1) 选择作业
- (2) 分配资源
- (3) 创建进程
- (4) 作业控制
- (5) 回收资源



CPU的三级调度

2. 中级调度（年级主任）

又称交换调度，功能是在内存使用紧张的情况下，将内存中暂时无法运行的进程挂起，即由内存调至外存（换出），使外存上具备运行条件的就绪进程能够及时进入内存运行。

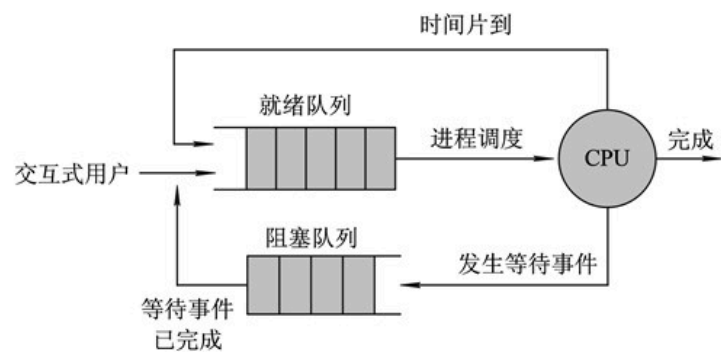
3. 低级调度（班主任）

又称进程调度或微观调度，功能是按照一定的调度算法将CPU分派给就绪队列中的某个进程。

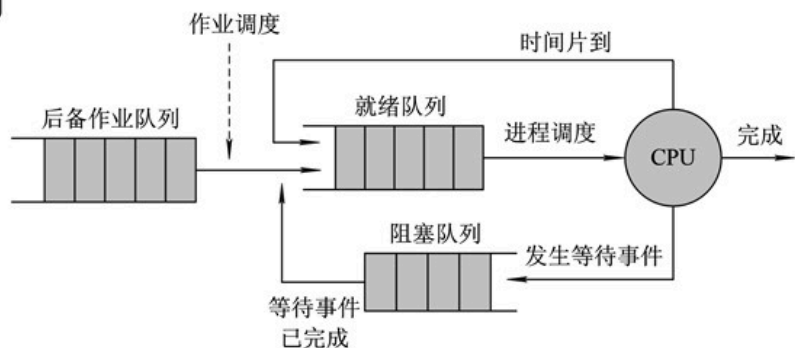
2.3 CPU调度策略

- 处理器调度队列模型

- 1. 仅有进程调度的调度队列模型

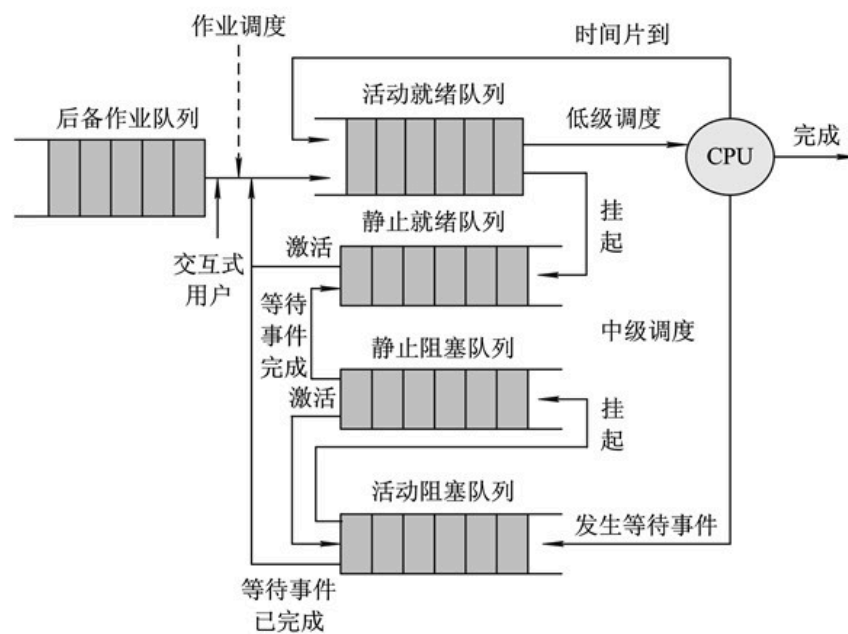


- 2. 具有高级和低级调度的调度队列模型



2.3 CPU调度策略

3. 同时具有三级调度的调度队列模型



单选题 1分

操作系统中的三级调度是指_____。

- ☐ A CPU 调度、资源调度和网络调度
- ☐ B CPU 调度、设备调度和存储器调度
- ☐ C 作业调度、进程调度和资源调度
- ☐ D 作业调度、进程调度和中级调度

2.3 CPU调度策略

- 进程调度的方式和时机

- 1. 进程调度方式

- 分为非抢占式（非剥夺式）调度和抢占式（剥夺式）调度。
- （1）非抢占式调度；（2）抢占式调度。

- 2. 进程调度的时机

有两个：（1）非抢占，当前运行进程执行结束而终止，或因等待某个事件的完成而无法继续执行，需要启动进程调度程序来选择一个新的就绪进程运行；

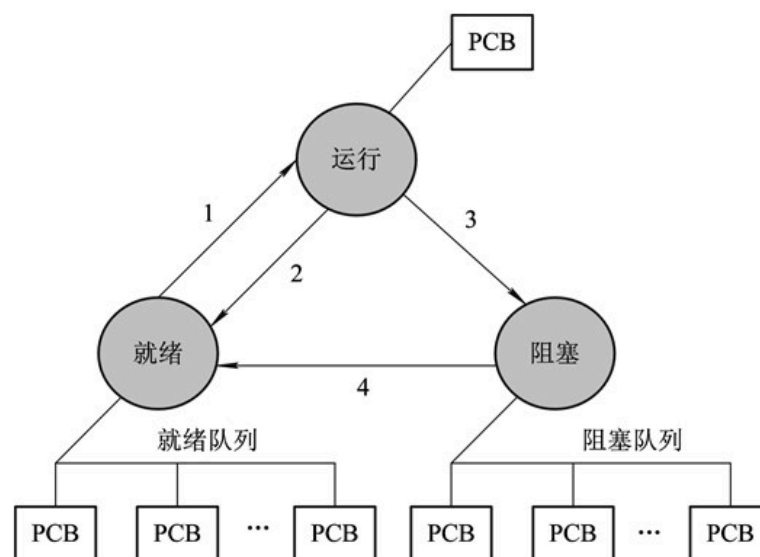
（2）在抢占式调度系统中，就绪队列中出现了优先级更高的进程，或当前运行进程的时间片已经用完，需要剥夺当前运行进程的CPU使用权，并将其分配给更高优先级的进程或时间片轮转的下一个就绪进程。

- 3. 进程调度实现

- （1）保存当前运行进程的现场信息
- （2）选择待运行的进程
- （3）为新选中的进程恢复现场

填空题 4分

在单CPU系统中，进程三种基本状态变迁如图所示。如下的几种状态变迁，哪种会发生？ (1) 2→1 [填空1]； (2) 3→2 [填空2]； (3) 4→1 [填空3]； (4) 3→1 [填空4]



单选题 1分

【全国统考真题】一个进程的读磁盘操作完成后，OS针对该进程必做的是（ ）。

- ☐ A 修改进程状态为就绪状态
- ☐ B 降低进程优先级
- ☐ C 为进程分配用户内存空间
- ☐ D 延长进程的时间片

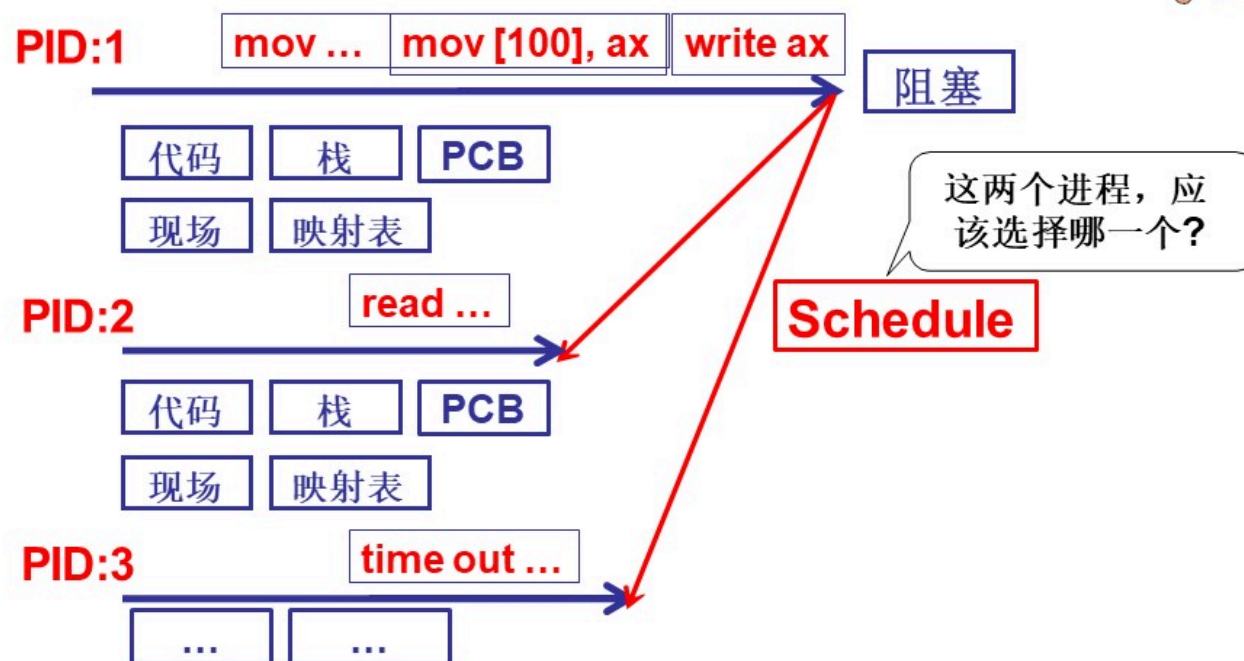
单选题 1分

在单CPU 的多进程系统中，进程切换时什么时候占用CPU以及占用多长时间取决于_____

- ☐ A 进程相应程序段的长度
- ☐ B 进程总共需要运行时间的多少
- ☐ C 进程自身和进程调度策略
- ☐ D 进程完成什么功能

2.3 CPU调度策略

多进程图像与CPU调度



CPU调度(进程调度)的直观想法



■ FIFO?

- 谁先进入，先调度谁: 简单有效 银行、食堂
- 一个只简单询问业务的人该怎么办?

■ Priority?

- 任务短可以适当优先 但你怎么知道这个任务将来会执行多长时间呢?
- 这人的询问越来越长怎么办?
- 那如果一个银行业务很长是因为客户需要填写一个很长的表，该怎么办?

应该还有很多这样的询问...

面对诸多场景，如何设计调度算法？

■ 我们的算法应该让什么更好？

- 面对客户：银行调度算法的设计目标应该是 **用户满意**
 - 面对进程：**CPU**调度的目标应该是 **进程满意**
 - 那怎么才能让进程满意呢？ **时间...**
 - 尽快结束任务：周转时间(从任务进入到任务结束)短
 - 用户操作尽快响应：响应时间(从操作发生到响应)短
 - 系统内耗时间少：吞吐量(完成的任务量)
- 总原则：**时间。系统专注于任务执行，又能合理调配任务...**

1. 面向系统的准则

(1) 吞吐量：系统单位时间内完成工作的一种度量。

(2) CPU利用率： $\text{CPU 利用率} = \frac{\text{CPU 有效工作时间}}{\text{CPU 总的运行时间}}$

(3) 系统资源平衡利用

(4) 公平性

2. 面向用户的准则

(1) 周转时间：指该作业由提交到完成所花费的时间，即：

作业*i*周转时间 T_i = 作业*i*完成时间 - 作业*i*提交时间

也可表示为：

作业*i*周转时间 T_i = 作业*i*运行时间 + 作业*i*等待时间

作业带权周转时间：

$$W_i = \frac{\text{作业 } i \text{ 周转时间 } T_i}{\text{作业 } i \text{ 运行时间}}$$

N个作业的平均周转时间：

$$T = \frac{1}{N} \sum_{i=1}^N \text{作业 } i \text{ 周转时间 } T_i$$

N个作业的平均带权周转时间：

$$W = \frac{1}{N} \sum_{i=1}^N \text{作业 } i \text{ 带权周转时间 } W_i$$

(2) 响应时间：从用户通过键盘提交一个请求开始，直到系统首次产生响应为止的时间间隔。

(3) 截止时间：是实时系统选择调度算法的重要准则。可以是某实时任务（作业或进程）必须开始的最迟时间，或是某实时任务必须完成的最迟时间。

(4) 优先权准则。

单选题 1分

现有3 个同时到达的作业J1、J2 和J3，它们的执行时间分别为T1、T2 和T3，且 $T1 < T2 < T3$ 。系统按单道方式运行且采用短作业优先算法，则平均周转时间是

- A $T1+T2+T3$
- B $(T1+T2+T3)/3$
- C $(3T1+2T2+T3)/3$
- D $(T1+2T2+3T3)/3$

如何做到合理？ 需要折中， 需要综合...

■ 吞吐量和响应时间之间有矛盾...

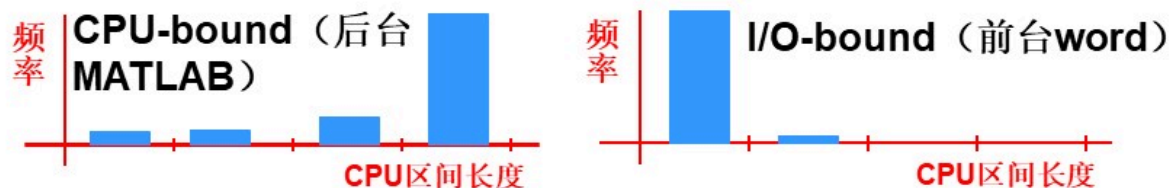
这样的矛盾还有很多...

- 响应时间小 $\Rightarrow$ 切换次数多 $\Rightarrow$ 系统内耗大 $\Rightarrow$ 吞吐量小

■ 前台任务和后台任务的关注点不同...

- 前台任务关注响应时间， 后台任务关注周转时间

■ IO约束型任务和CPU约束型任务有各自的特点



折中和综合让操作系统变得复杂， 但有效的系统又要求尽量简单...



各种CPU调度算法

常用调度算法

1. 先来先服务调度算法（FCFS）

高级调度时，FCFS调度算法按照作业进入后备作业队列的先后顺序选择作业进入内存，即先进入后备作业队列的作业被优先选择进入内存，然后为选中的作业创建进程并分配该作业所需资源。

低级调度时，FCFS调度算法每次从内存的进程/线程就绪队列中选择一个最先进入的进程/线程，然后由进程调度程序将CPU分配给它并使其运行。

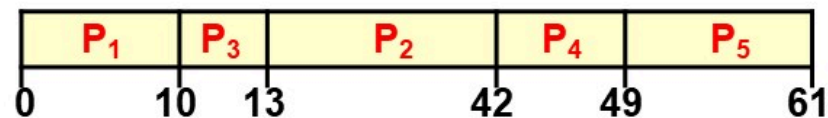
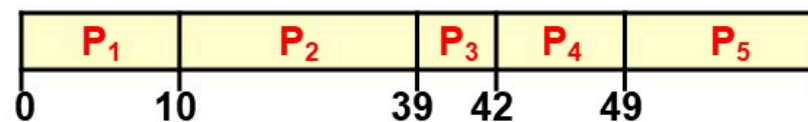
优点：实现简单，公平性

缺点：没有考虑作业的类型或进程/线程执行时间的长短，短作业或I/O进程/线程等待时间过长。

各种CPU调度算法

First Come, First Served (FCFS)

任务	到达时间	CPU区间(ms)
P ₁	0.001	10
P ₂	0.002	29
P ₃	0.003	3
P ₄	0.004	7
P ₅	0.005	12



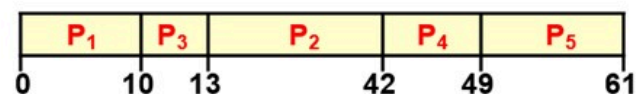
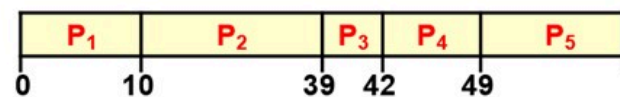
填空题 2分

FCFS平均周转时间 [填空1]

P2和P3调换一下, 平均周转时间 [填空2]

First Come, First Served (FCFS)

任务	到达时间	CPU区间(ms)
P ₁	0.001	10
P ₂	0.002	29
P ₃	0.003	3
P ₄	0.004	7
P ₅	0.005	12

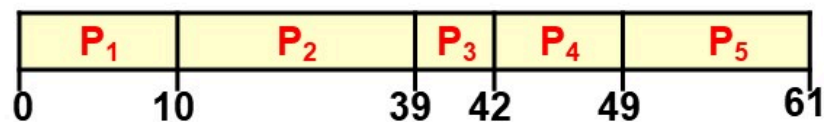


各种CPU调度算法

First Come, First Served (FCFS)

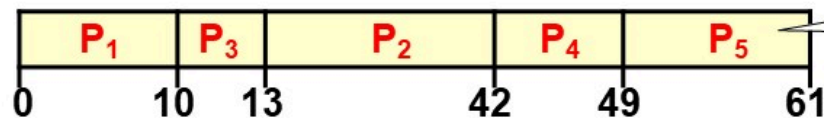
任务	到达时间	CPU区间(ms)
P ₁	0.001	10
P ₂	0.002	29
P ₃	0.003	3
P ₄	0.004	7
P ₅	0.005	12

平均周转时间：
 $(10+39+42+49+61)/5 = 40.2$



平均周转时间：

$$(10+13+42+49+61)/5=35$$



如何缩短周转时间?

SJF: 短作业优先



■ P_3 提前的完成时间较 P_2 拖后的完成时间要小

■ 如果调度结果是 $p_1p_2...p_n$, 则
平均周转时间为:

$$p_1 + p_1 + p_1 + p_1 + p_1 + p_2 + p_2 + p_2 + \dots = \sum (n+1-i)p_i$$

如果存在 $i < j$ 而 $p_i > p_j$, 交换 p_i, p_j
会怎么样?

平均周转时间?

$$(3+13+20+32+61)/5=25.2$$

如何缩短周转时间?

常用调度算法

2. 短作业/短进程优先调度算法 (SJF/SPF)

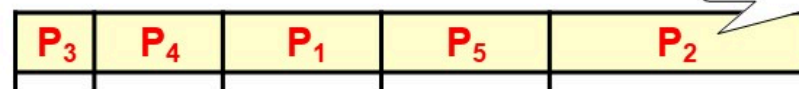
每次从后备作业队列中选择估计运行时间最短的作业进入内存，并创建相应的进程。

SJF/SPF调度算法是一种非抢占式调度算法，某作业的进程一旦获得了CPU，就一直运行到进程完成或因某事件阻塞而放弃CPU。所以，SJF/SPF调度算法不适合于分时系统或实时系统。

优点：能有效地降低作业/进程的平均等待时间，提高系统的吞吐量。

缺点：用户提供的估计运行时间不一定准确，此外长作业/长进程有可能长时间等待而得不到运行。

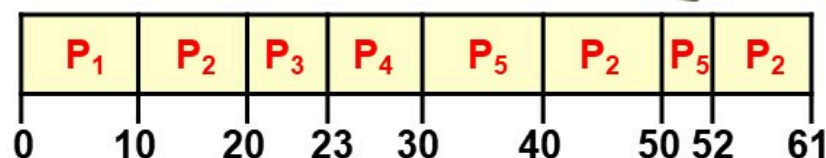
响应时间该怎么办?



P₂用户的操作...

■ 如何解决? 开动脑筋...

■ **RR: 按时间片来轮转调度**



- 时间片大: 响应时间太长; 时间片小: 吞吐量小
- 折衷: 时间片**10-100ms**, 切换时间**0.1-1ms(1%)**

■ 如果**CPU**更快, 时间片会怎么样?

响应时间该怎么办?

3. 时间片轮转调度算法 (RR)

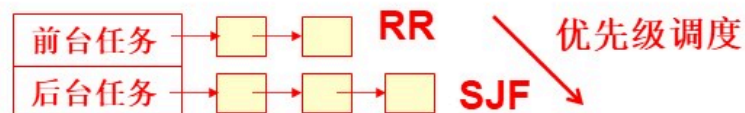
进程/线程就绪队列按进程/线程到达系统时间的先后次序进行排队，进程调度程序按先来先服务的原则选择就绪队列中的第一个进程/线程，将CPU分配给它执行。

进程/线程每次使用CPU的时间只能是一个时间片，当运行进程/线程用完规定的时间片时必须放弃CPU的使用权；这时，进程调度程序又将CPU分配给当前就绪队列的第一个进程，而放弃CPU的进程则回到就绪队列的队尾，等待下次轮到自己时再投入运行。

时间片轮转调度算法的核心是时间片。

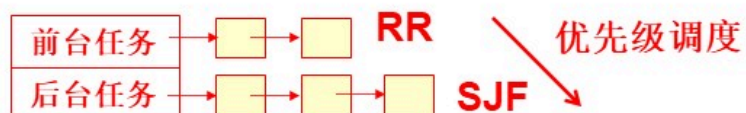
响应时间和周转时间同时存在，怎么办？

- **Word**很关心响应时间，而**gcc**更关心周转时间，两类任务同时存在怎么办？
- 一个调度算法让多种类型的任务同时都满意，怎么办？
- **直观想法**：定义前台任务和后台任务两队列，前台**RR**，后台**SJF**，只有前台任务没有时才调度后台任务



- 但是这会产生很多问题？想一想会出现什么问题？

如果一直有前台任务...



一个故事: 1973年关闭的MIT的IBM 7094时, 发现有一个进程在1967年提交但一直未运行

- 后台任务可能一直得不到运行
- 后台任务优先级动态升高, 但后台任务(用SJF调度)一旦执行, 前台的响应时间...
- 前后台任务都用时间片, 但又退化为了RR, 后台任务的SJF如何体现? 前台任务如何照顾?

问题: 分析到此处了, 我们该怎么办?

还有很多问题...

- 我们怎么知道哪些是前台任务，哪些是后台任务，**fork** 时告诉我们吗？
- **gcc**就一点不需要交互吗？**Ctrl+C**按键怎么工作？**word**就不会执行一段批处理吗？**Ctrl+F**按键？
- **SJF**中的短作业优先如何体现？如何判断作业的长度？这是未来的信息...

单选题 1分

在进程调度算法中，对短进程不利的是_____

- ☐ A 短进程优先调度算法
- ☐ B 先来先服务算法
- ☐ C 高响应比优先算法
- ☐ D 多级反馈队列调度算法

常用调度算法

4. 高响应比优先调度算法（HRRF）

是一种基于动态优先数的非抢占式调度算法。

按照高响应比优先调度算法，每个作业或进程/线程都拥有一个动态优先数，该优先数不仅是作业或进程/线程运行时间（估计值）的函数，还是其等待时间的函数。

$$R_p = \frac{\text{响应时间}}{\text{运行时间}} = \frac{\text{运行时间} + \text{等待时间}}{\text{运行时间}} = 1 + \frac{\text{等待时间}}{\text{运行时间}}$$

优点：既照顾了短作业/短进程，而又不使长作业/长进程等待时间过长，是先来先服务调度算法和短作业/短进程优先调度算法的一种折中调度方案。

缺点：需要估计每个作业或进程/线程的运行时间，而且每次调度时都要计算后备作业队列中所有作业或就绪队列中所有进程的响应比，这需要耗费不少的CPU时间。

常用调度算法

5. 优先级调度算法

既可用于高级调度，也可用于低级调度，还可用于实时系统。

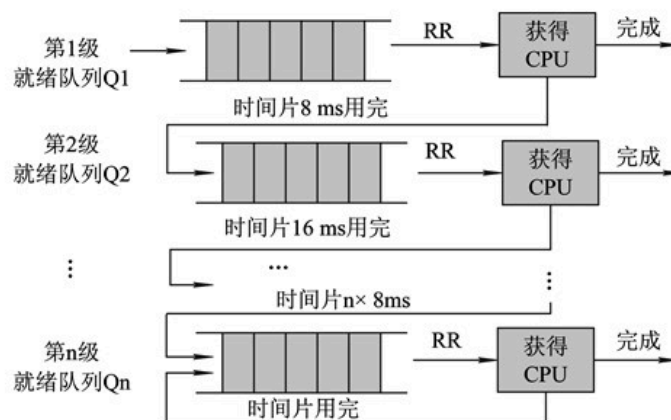
若调度的对象为作业，优先级调度算法每次从后备作业队列中选择优先级最高的作业调入内存，为其分配相应的资源并创建进程放入到就绪队列。

若调度的对象为进程，则优先级调度算法每次从就绪队列中选择优先级最高的进程为其分配CPU而投入运行。如果有多个优先级最高的作业/进程，则可结合先来先服务或短作业/短进程优先调度策略。

优点：公平性好、灵活、资源利用率高

常用调度算法

6. 多级反馈队列调度算法 (MLFQ)



优点：短进程能够得到优先处理；系统开销不大；交互型请求通常能够在第一个就绪队列中完成。

缺点：如果优先级较高的队列一直不为空，则优先级较低队列中的进程可能长时间无法得到运行，即会导致发生饥饿现象。

常用调度算法

实时调度

是为了完成实时处理任务而分配CPU的调度方法，它的基本要求是保证计算机在规定的时间内对外部事件的请求做出响应。

实时调度与非实时调度的主要区别是：

- (1) 实时调度所调度的任务有完成时限，而非实时调度则没有。实时调度算法的正确与否与算法的逻辑和算法调度的时限有关。
 - (2) 实时调度要求有较快的进程/线程切换时间，而非实时调度的进程/线程切换时间较长。
 - (3) 非实时调度强调资源利用率（批处理系统）或用户共享CPU（分时系统），实时调度则主要强调在规定的时限范围内完成对相应对象的控制。
 - (4) 实时调度为抢占式调度，而非实时调度则很少采用抢占式调度。
- 优点：短进程能够得到优先处理；系统开销不大；交互型请求通常能够在第一个就绪队列中完成。
- 缺点：如果优先级较高的队列一直不为空，则优先级较低队列中的进程可能长时间无法得到运行，即会导致发生饥饿现象。

常用调度算法

实时调度

分类：

(1) 比率单调 (Rate Monotonic Algorithm) 调度算法

算法的任务优先级是按照任务周期来确定的。那些具有短执行周期的任务具有较高的优先级，而周期长的任务优先级较低。调度程序总是将CPU分配给当前优先级最高的进程/线程，且进程/线程执行过程中允许更高优先级的新进程/线程抢占当前运行进程/线程所使用的CPU。

比率单调调度算法实现简单、系统开销小、灵活性较好，是实时调度的基础性算法，缺点是CPU的利用率低。

(2) 最早截止时间优先调度算法

根据任务的截止时间来动态确定任务的优先级，截止时间越短则优先级越高。这种调度算法更多用于抢占式调度。

(3) 最短空闲时间优先调度算法

在进行调度时，任务的优先级根据任务的空闲时间动态确定，任务的空闲时间越短则任务的优先级越高。

单选题 1分

下列选项中，满足短进程优先且不会发生饥饿现象的调度算法是_____

- ☐ A 先来先服务
- ☐ B 响应比高者优先
- ☐ C 时间片轮转
- ☐ D 非抢占式短进程优先

单选题 1分

一个作业8:00到达系统，估计运行时间为1小时。若从10:00开始执行该作业，其响应比是____

- ☐ A 2
- ☐ B 1
- ☐ C 3
- ☐ D 0.5

填空题 6分

假设有四个作业，基本信息如下表：

分别采用FCFS、SJF、优先级算法（规定优先数越小优先级越高）。给出调度顺序，计算在单道程序环境下的平均周转时间和平均带权周转时间。

FCFS平均周转时间 [填空1] FCFS平均带权周转时间 [填空2]

SJF平均周转时间 [填空3] SJF平均带权周转时间 [填空4]

优先级平均周转时间 [填空5] 优先级平均带权周转时间 [填空6]

作业号	提交时间	估计运行时间	优先数
1	10.0	2.0	2
2	10.2	1.0	4
3	10.4	0.5	1
4	10.5	0.3	3

解答要点:

周转时间=完成时间-提交时间 带权周转时间=周转时间/ 执行时间

FCFS 调度顺序: 1→2→3→4

作业号	提交时间	估计运行时间	开始执行时间	结束执行时间	周转时间	带权周转时间
1	10.0	2.0	10.0	12.0	2.0	2/2=1.0
2	10.2	1.0	12.0	13.0	13.0-10.2=2.8	2.8/1.0=2.8
3	10.4	0.5	13.0	13.5	13.5-10.4=3.1	3.1/0.5=6.2
4	10.5	0.3	13.5	13.8	13.8-10.5=3.3	3.3/0.3=11.0
平均					2.8	5.25

SJF 调度顺序: 1→4→3→2 (非剥夺式)

作业号	提交时间	估计运行时间	开始执行时间	结束执行时间	周转时间	带权周转时间
1	10.0	2.0	10.0	12.0	2.0	$2/2=1.0$
2	10.2	1.0	12.8	13.8	$13.8-10.2=3.6$	$3.6/1.0=3.6$
3	10.4	0.5	12.3	12.8	$12.8-10.4=2.4$	$2.4/0.5=4.8$
4	10.5	0.3	12.0	12.3	$12.3-10.5=1.8$	$1.8/0.3=6$
平均					2.45	3.85

优先级法调度顺序: 1→3→4→2 (非剥夺式)

作业号	提交时间	估计运行时间	优先数	开始执行时间	结束执行时间	周转时间	带权周转时间
1	10.0	2.0	2	10.0	12.0	2.0	$2/2=1.0$
2	10.2	1.0	4	12.8	13.8	$13.8-10.2=3.6$	$3.6/1.0=3.6$
3	10.4	0.5	1	12.0	12.5	$12.5-10.4=2.1$	$2.1/0.5=4.2$
4	10.5	0.3	3	12.5	12.8	$12.8-10.5=2.3$	$2.3/0.3=7.7$
平均						2.5	4.125

2.4 进程控制

进程切换

实质是回收当前运行进程对CPU的控制权，并将CPU控制权转交给新调度的进程这样一个过程。

进程的上下文——一个进程运行时，CPU所有寄存器中的内容、进程的状态以及运行栈中的内容。

进程上下文的组成：

- (1) 系统级上下文
- (2) 寄存器上下文
- (3) 用户级上下文

进程切换

- 进程切换的时机

- (1) 中断
- (2) 异常
- (3) 系统调用

- 进程上下文切换

- (1) 当前运行进程被中断时保存其CPU现场信息。
- (2) 对被中断的当前运行进程进行PCB更新，包括改变进程状态和其他相关信息。
- (3) 将被中断的当前运行进程的PCB移入适当的队列（时间片到则移入就绪队列，因某事件阻塞则移入相应的阻塞队列）。
- (4) 由进程调度程序调度选中另一个就绪进程，为其设置执行的上下文环境并对其PCB进行更新。
- (5) 修改新进程的地址空间，更新新进程的内存管理信息。
- (6) 恢复被选中进程最后一次进程上下文切换时所保存的CPU现场信息。

2.4 进程控制

进程控制原语

1. 原语

一个操作可以依次分成几个具体实施的动作，如果这几个动作的执行不会被分割或中断，且这些动作要么全部执行，要么一个都不执行，则称这个操作具有原子性。

一个特殊的程序段称为原语。

2. 进程控制原语主要有创建、撤销、阻塞和唤醒等。

进程控制原语

1. 进程的创建原语

```
int Creat(int n, int k, List *s, List *r) //创建原语
{
    lock out interrupts;           //关中断
    Get(NewPCB, FreeQueue); //从空闲PCB队列FreeQueue中申请一个空闲PCB
    NewPCB->id=n;                 //为新进程分配唯一的进程标识符n
    NewPCB->priority=k;           //置新进程的优先级为k
    GetMemory(NewPCB);           //为新进程申请分配内存资源
    GetResoure(NewPCB, r);       //按资源清单r为新进程分配其他资源
    Init_PCB(NewPCB, s);         //按初始化清单s初始化新进程的PCB
    NewPCB->status="ready";      //新进程的状态置为“就绪”
    NewPCB->parents=EP();         //新进程的父进程为当前执行进程
    NewPCB->children=NULL;       //新进程的子进程初始为空
    Insert(ReadyQueue, NewPCB); //将新进程的PCB插入到就绪队列ReadyQueue中
    unlock interrupts;           //开中断
    return(n);                   //将新进程的标识符n返回给当前执行进程
}
```

进程控制原语

2. 进程的撤销原语

```
void Destroy(string name)      //撤销原语
{lock out interrupts;         //关中断
int i=0, dispatch=0;          //置重新调度标志dispatch的初值为假
i=Search(PCB_List, name);     //根据进程名name在PCB链中查找其进程标识符
if(i==0)                      // i等于0则未找到要撤销进程的PCB
    Unusual_Stop();           //转异常终止处理
else                          //找到要撤销进程的PCB
    if(i.status=="excute")    //被撤销的进程i是运行进程
    {    dispatch=1;          //置重新调度标志为真
        stop(i);              //终止进程i的执行
    }
    else
    {    Destroy(Progeny(i));  //撤销进程i的所有子孙进程
        free(i.resoure);      //释放进程i占有的资源
        Remove(i);            //从就绪队列或阻塞队列中移去进程i的PCB
        Insert(FreeQueue, i); //将进程i的PCB插入到空闲PCB队列
    }
if(dispatch==1)               //当重新调度标志为真时
{    unlock interrupts;       //开中断
    Scheduler();              //执行进程调度程序调度一就绪进程运行
}
else    unlock interrupts;    //开中断
}
```

进程控制原语

3. 进程的阻塞原语

```
void Block()                //阻塞原语
{
    lock out interrupts;    //关中断
    i=Ep();                 //获得当前运行进程的进程标识符i
    Store(i_PCB);           //将i的CPU现场信息暂存于i的PCB中
    i.status="block";       //置i的状态为“阻塞”
    Insert(BlockQueue, i);  //将进程i的PCB插入到阻塞队列BlockQueue中
unlock interrupts;          //开中断
Scheduler();                //执行进程调度程序调度一就绪进程运行
}
```

进程控制原语

4. 进程的唤醒原语

```
void Wakeup(List r)           //唤醒原语
{
    lock out interrupts;      //关中断
    i=Search(BlockQueue, r);  //根据已释的资源r查找r阻塞队列的队首进程
    Remove(i);                //从r阻塞队列的队首移出进程i
    GetResoure(i_PCB, r);      //将资源r分配给进程i
    i. status="ready";        //置i的状态为“就绪”
    insert(ReadyQueue, i);     //将进程i的PCB插入到就绪队列
unlock interrupts;           //开中断
}
```

进程控制原语

5. 进程的挂起原语

挂起原语的执行过程：

- (1) 根据被挂起进程的标识号从PCB集合中查找出该进程的PCB，从中读出该进程的状态。
- (2) 修改PCB中的进程状态。若该进程处于就绪（活动就绪）状态就将它修改为挂起就绪状态；若处于阻塞（活动阻塞）状态就将它修改为挂起阻塞状态；若处于运行状态就将它修改为挂起就绪状态。
- (3) 将被挂起进程的PCB复制到指定的内存区域，以便用户或父进程考察该进程的运行情况。
- (4) 将被挂起进程的非常驻部分从内存换出至外存的对换区。
- (5) 若被挂起进程挂起前为执行态（正在执行），则执行进程调度程序从就绪队列中选中一个进程执行。

进程控制原语

5. 进程的激活原语

激活原语的执行过程：

- (1) 将被激活的进程由外存调入到内存。
- (2) 修改该进程的状态。若该进程的现行状态是挂起就绪则将其修改为就绪（活动就绪）状态；若该进程的现行状态是挂起阻塞则将其修改为阻塞（活动阻塞）状态。
- (3) 若系统采用抢占式调度策略且被激活的进程其状态为就绪状态，还需比较被激活进程与当前正在运行进程的优先级，如果被激活进程的优先级高则需重新调度，即暂停当前运行的进程执行而将CPU分配给刚唤醒的进程。

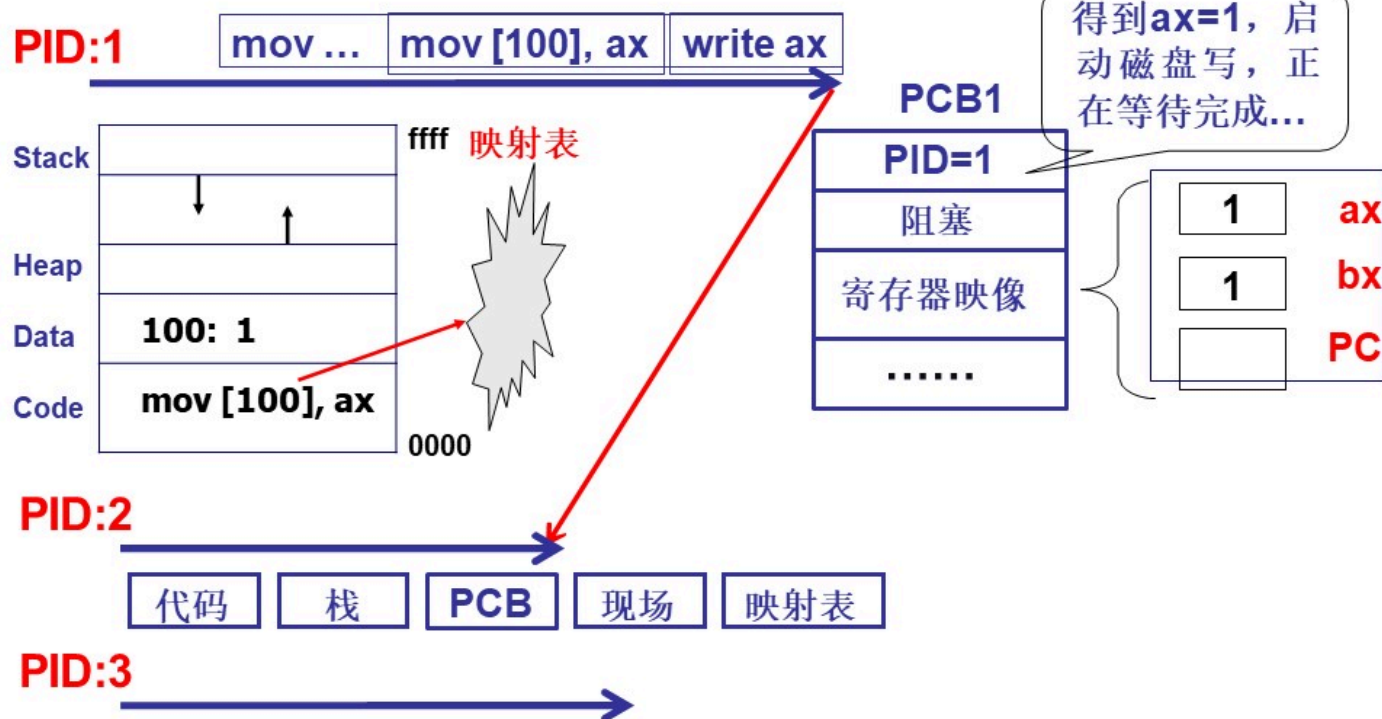
单选题 1分

进程自身决定_____。

- ☐ A 从运行状态到阻塞状态
- ☐ B 从运行状态到就绪状态
- ☐ C 从就绪状态到运行状态
- ☐ D 从阻塞状态到就绪状态

2.5 线程

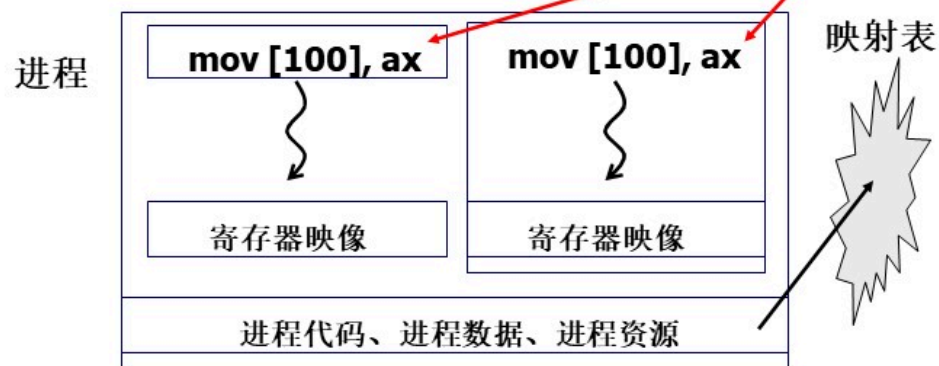
多进程是操作系统的基本图像



是否可以资源不动而切换指令序列?

■ **进程 = 资源 + 指令执行序列**

- 将资源和指令执行分开
- 一个资源 + 多个指令执行序列



■ **线程**: 保留了并发的优点, 避免了进程切换代价

■ **实质就是映射表不变, 而PC指针变**

2.5 用户级线程

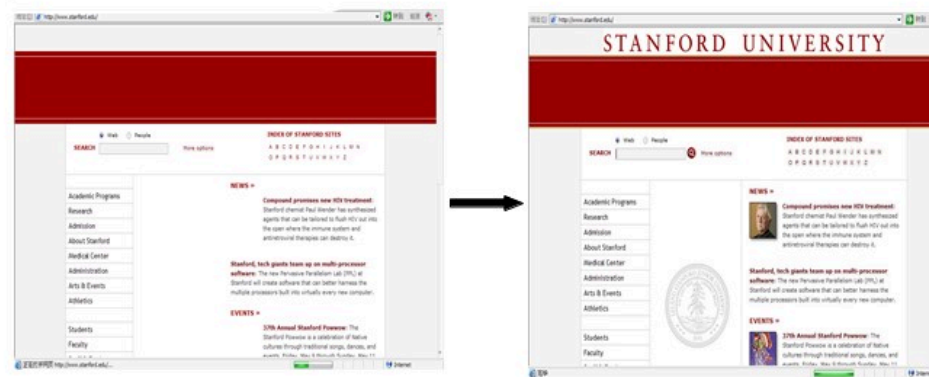
多个执行序列+一个地址空间是否实用？

■ 一个网页浏览器

- 一个线程用来从服务器接收数据
- 一个线程用来显示文本
- 一个线程用来处理图片(如解压缩)
- 一个线程用来显示图片

■ 这些线程要共享资源吗？

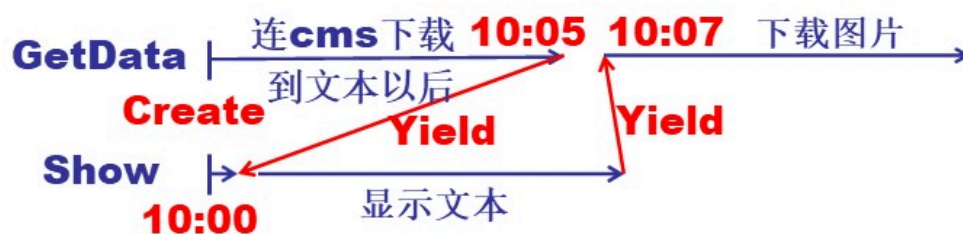
- 接收数据放在100处，显示时要读..
- 所有的文本、图片都显示在一个屏幕上



开始实现这个浏览器...

```
void WebExplorer()
{
    char URL[] = "http://cms.hit.edu.cn";
    char buffer[1000];
    pthread_create(..., GetData, URL, buffer);
    pthread_create(..., Show, buffer);
}

void GetData(char *URL, char *p){...}; void
Show(char *p){...};
```



Create? Yield?

■核心是Yield...

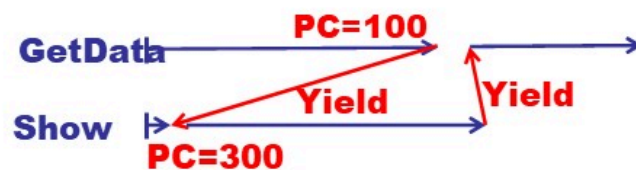
- 能切换了就知道切换时需要是个什么样子
- Create就是要制造出第一次切换时应该的样子

样子弄明白了，剩下的就是写程序实现这个样子了...

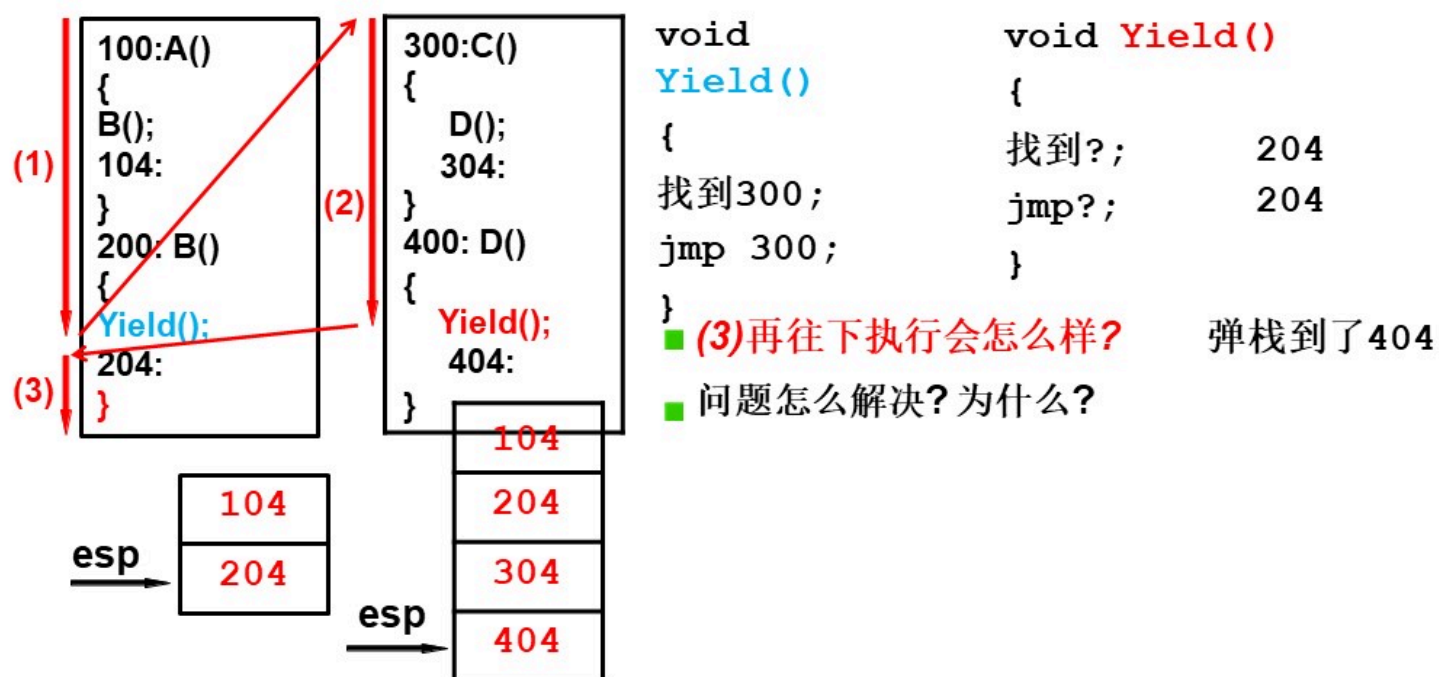
```
100:A()
{
  B(); 104:
}
200: B()
{
  Yield();
204:
}
```

```
300:C()
{
  D(); 304:
}
400: D()
{
  Yield();
404:
}
```

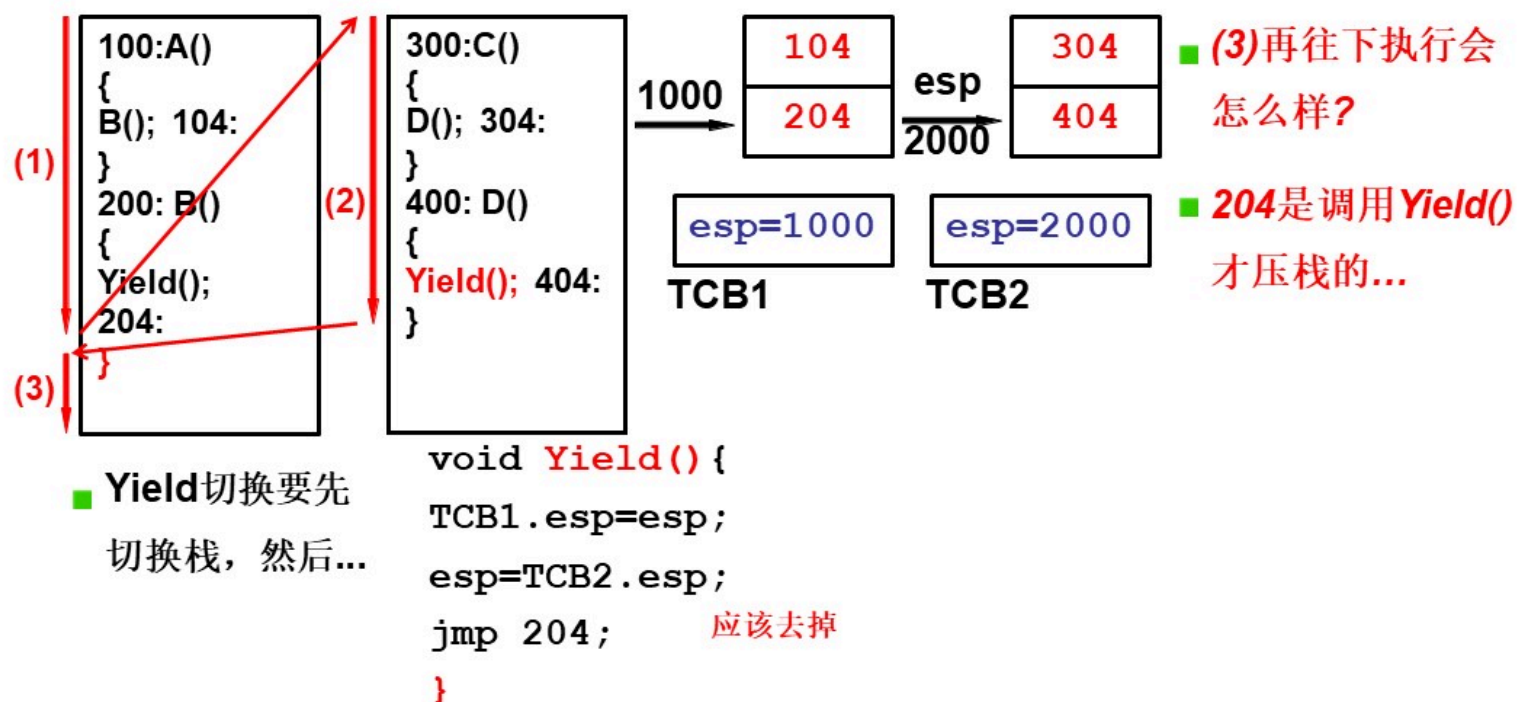
■仔细看Yield，就是100跳到300



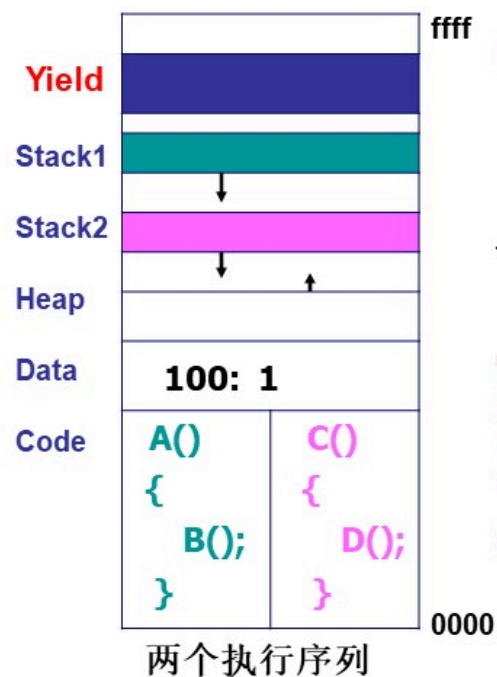
两个执行序列与一个栈...



从一个栈到两个栈...



两个线程的样子：两个TCB、两个栈、切换的PC在栈中



■ ThreadCreate的核心就是用程序做出这三样东西



```
void ThreadCreate (A)
{
    TCB *tcb=malloc(); //申请内存作为TCB
    *stack=malloc(); //申请内存作为栈
    *stack = A; //100 栈里面填上内容
    tcb.esp=stack; //栈和TCB关联
}
```

将所有东西组合在一起.....

```
void WebExplorer() //main()
{ ThreadCreate(GetData, URL, buffer); ...
while(1) Yield(); }
```

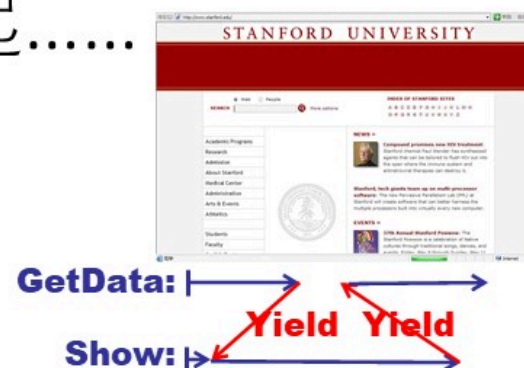
```
void GetData(char *URL, char *p) {
连接URL; 下载; Yield(); ... }
```

```
void ThreadCreate(func, arg1) {
申请栈; 申请TCB; func等入栈; 关联TCB与栈; ... }
```

```
void Yield() { 压入现场; esp放在当前TCB
中; Next(); 从下个TCB取出esp; 弹栈切换线程; }
```

调度函数，对系统影响很大，如可优先调度show!

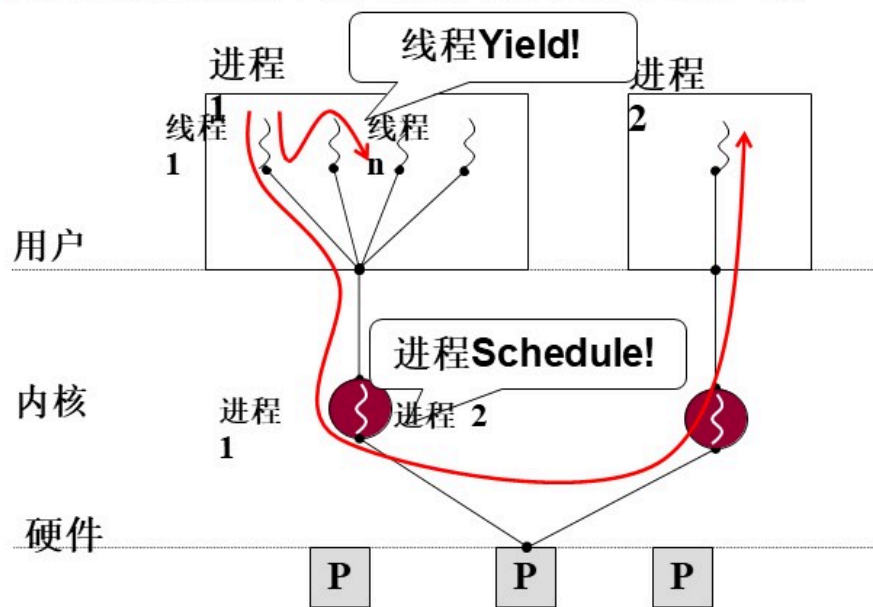
■ gcc -o explorer get.c yield.c ... 或 gcc get.c.. -lthread



■ `GetData`下载到文本时会调用`Yield()`...

为什么说用户级线程——Yield是用户程序

■ 如果进程的某个线程进入内核并阻塞，则...



GetData

连接URL发起请求;
等待网卡IO... 进程阻塞

Show

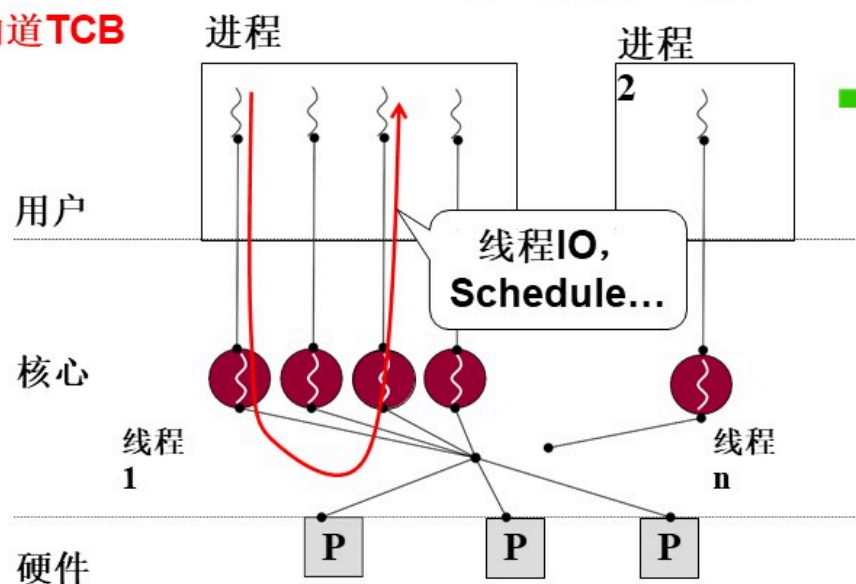
显示文本和链接;
...;

内核级线程和用户级线程区别，哪个快？

内核级线程

- **ThreadCreate** 是系统调用，会进入内核，内核

知道TCB
1



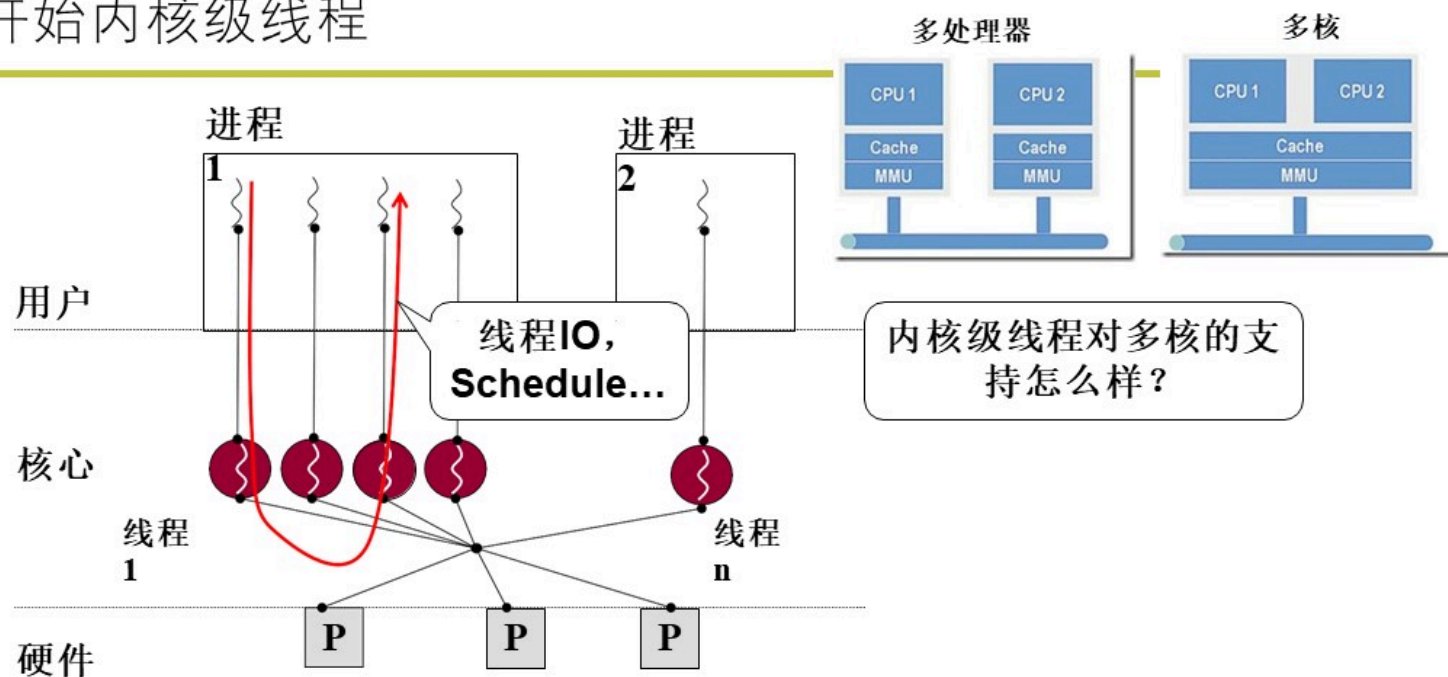
- `gcc -o explorer explorer.c yield.c ...`

- 内核级线程 `gcc -o explorer explorer.c...`;

ThreadCreate 是系统调用;
Yield() 用户不可见，调度点由系统决定

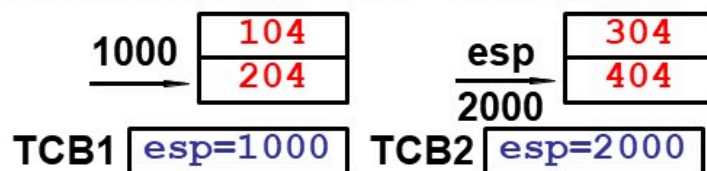
2.6 内核级线程

开始内核级线程

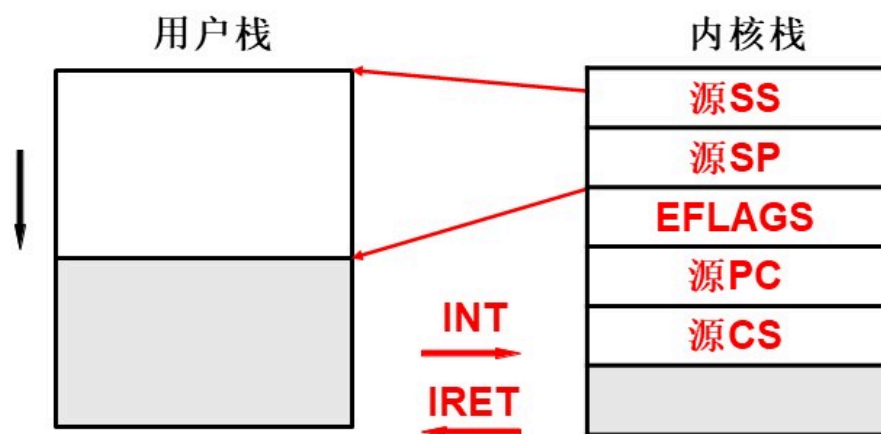


和用户级相比，核心级线程有什么不同？

- **ThreadCreate**是系统调用，内核管理**TCB**，内核负责切换线程
- 如何让切换成型？– 内核栈，**TCB**
 - 用户栈是否还要用？执行的代码仍然在用户态，还要进行函数调用
 - 一个栈到一套栈；两个栈到两套栈
 - **TCB**关联内核栈，那用户栈怎么办？

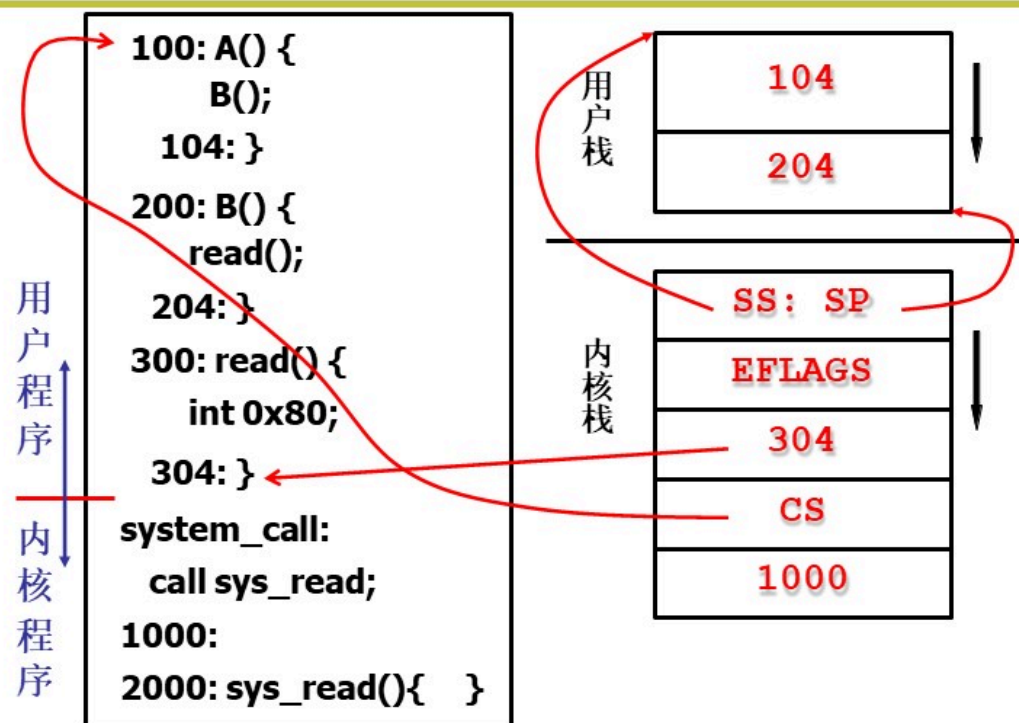


用户栈和内核栈之间的关联



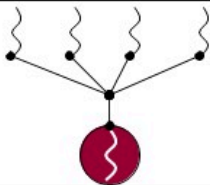
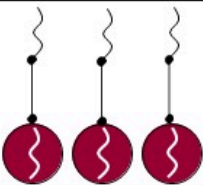
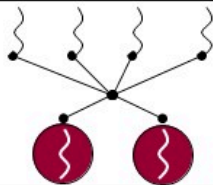
- 所有中断(时钟、外设、INT 指令)都引起上述切换
- 中断(硬件)又一次帮助了操作系统...

仍然是那个A(), B(), C(), D()...



■ 认真体会从内核返回
(中断返回)时的样子...

用户级线程、核心级线程的对比

	用户级线程	核心级线程	用户+核心级
实现模型			
利用多核	差	好	好
并发度	低	高	高
代价	小	大	中
内核改动	无	大	大
用户灵活性	大	小	大

单选题 1分

下面关于线程的叙述中，正确的是

- ☒ A 内核态线程的切换都需要内核的支持
- ☐ B 线程是资源的分配单位而进程是调度和分配的单位
- ☐ C 不管系统中是否有线程，线程都是拥有资源的独立单位
- ☐ D 在引入线程的系统中，进程仍然是资源分配和调度分派的基本单位

2.5 线程

线程的引入

操作系统必须的操作：

(1) 创建进程；(2) 进程切换；(3) 撤销进程

为了使多个程序能够更好地并发执行同时又尽量减少系统的开销，可将进程所承担的两个角色由操作系统分开，承担资源分配的实体不再作为独立运行的实体，而作为调度、分派基本单位的运行实体也不再是拥有资源的实体。

另一方面，多CPU计算机系统的出现为提高计算机的运行速度和吞吐量提供了良好的硬件条件，但要使各个CPU协调运行并充分发挥它们的并行处理能力，还必须配置良好的多CPU操作系统。

在多CPU系统中进程已不再适合作为独立运行的实体，而以线程作为独立运行单位则可充分发挥多个CPU的并行处理能力。

2.5 线程

线程的概念

线程是CPU调度和执行的最小单元。

- (1) 进程内的一个执行单元。
- (2) 进程内的一个可独立调度的实体。
- (3) 线程是进程中一个相对独立的控制流序列。
- (4) 线程是执行的上下文。

线程属性：

- (1) 线程属于轻型实体，基本不拥有系统资源，只拥有为保证其运行而必不可少的资源。
- (2) 线程是独立调度和分派的基本单位，也是能够独立运行的基本单位。
- (3) 同一个进程中的所有线程共享该进程所拥有的全部资源。
- (4) 线程并发执行程度高，不但同一个进程内部的多个线程可以并发执行，而且属于不同进程的多个线程也可以并发执行。

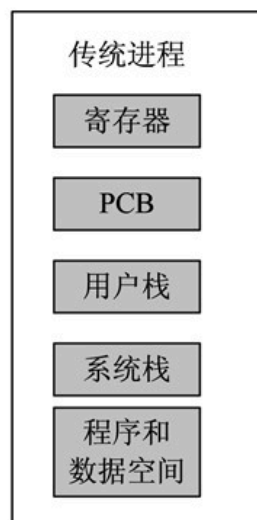
2.5 线程

线程的概念

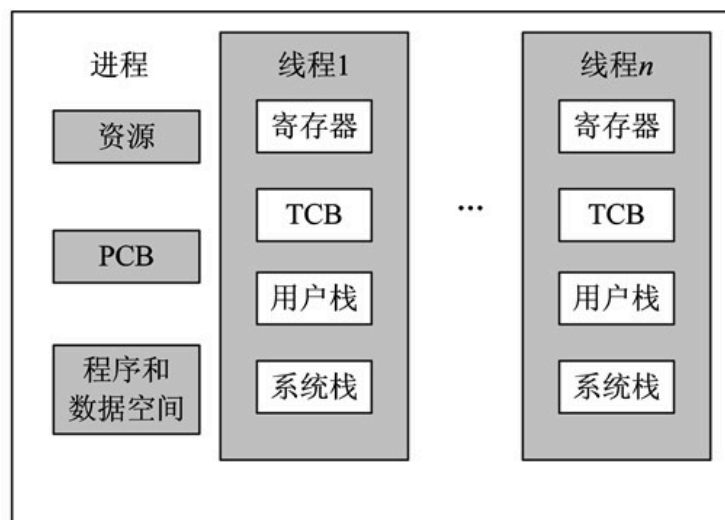
多线程操作系统提供的同步机制有以下几种：互斥锁、读写锁、条件变量和计数信号量等。这几种同步机制中：

- (1) 互斥锁仅允许每次使用一个线程来执行特定的代码或者访问特定的数据。
- (2) 读写锁允许对受保护的共享资源进行并发读取和独占写入。若要修改资源，线程必须首先获取互斥写锁；只有释放了所有的读锁之后，才允许使用互斥写锁。
- (3) 条件变量会一直阻塞线程，直到特定的条件为真。
- (4) 计数信号量通常用来协调对资源的访问，达到指定的计数时信号将阻塞；使用计数可以限制访问某个信号的线程数量。

线程与传统进程的比较



(a) 传统进程结构示意图



(b) 具有多线程的进程结构示意图

2.5 线程

线程与传统进程的比较

- (1) 传统进程除了是调度和分派的基本单位以外，还是资源分配的基本单位。而线程只是调度和分派的基本单位。
- (2) 在引入线程的系统中，同一个进程中的多个线程可以并发执行，且属于不同进程中的多个线程也可以并发执行，线程并发执行的程度高于传统进程并发执行的程度。
- (3) 创建、切换和撤销一个线程所花费的时空开销小于创建和撤销一个传统进程所花费的时空开销。
- (4) 传统进程是系统资源分配的基本单位，而线程基本不拥有资源，但可以使用它所隶属进程的资源，如程序段、数据段、打开的文件及I/O设备等。不同进程的地址空间是相互独立的，而属于同一个进程的所有线程共享同一个地址空间（该进程的地址空间）。
- (5) 由于不同进程具有各自独立的数据空间，数据传递通过通信方式进行，费时也不方便；在多线程操作系统中，一个进程的数据空间被该进程的所有线程所共享，一个线程的数据可以直接被属于同一个进程的其他线程所使用，因此数据传递既方便又快捷。

21. 试从调度、并发、拥有资源和系统开销这 4 个方面比较传统进程和线程。

【参考答案】传统进程与线程的比较如表1-2-1所示。

表1-2-1 传统进程与线程的比较

	调度	并发	拥有资源	系统开销
线程	线程是OS中调度和分派的基本单位，具有唯一的标识符和TCB	同一进程的多个线程可在一个/多个处理机上并发执行	同一进程的所有线程共享但不拥有进程的状态和资源，且驻留在进程的同一个内存地址空间中，可以访问相同的数据，通信和同步的实现也十分方便	作为系统调度和分派的基本单位，线程会被频繁地调度和切换；同一进程中的多线程共享同一地址空间，这能使线程实现快速切换
传统进程	传统进程具有独立的虚地址空间；以传统进程为单位进行任务调度时，系统必须切换地址空间，且切换时间长	许多“多任务OS”会限制用户拥有的最大进程数目，该数目对于许多并发应用来说是不够的	传统进程是系统中资源分配和保护的基本单位，也是系统调度的独立单位；每个传统进程都能以各自独立的速度在CPU上运行	对多个传统进程进行管理（如创建、调度、终止等）时，系统开销大，如在为了响应客户请求而须建立一个新的服务进程的服务器应用过程中，创建开销大

单选题 1分

【全国统考真题】下列关于进程和线程的叙述中，正确的是（）

- ☐ A 不管系统是否支持线程，进程都是资源分配的基本单位
- ☒ B 线程是资源分配的基本单位，进程是调度的基本单位
- ☐ C 内核支持线程和用户级线程的切换都需要内核的支持
- ☐ D 同一进程中的各个线程拥有各自不同的地址空间

单选题 1分

下面关于线程的叙述中，正确的是_____

- ☐ A 线程是比进程更小的能够独立运行的基本单位
- ☒ B 引入线程可以提高程序并发执行的程度，可以进一步提高系统的效率。
- ☐ C 线程的引入增加了程序执行的时空开销。
- ☐ D 一个进程一定包含多个线程

填空题 1分

(考研真题) 有 5 个进程 (见表 1-3-1) 需要调度执行, 若采用非抢占式优先级 (短进程优先) 调度算法, 问这 5 个进程的平均周转时间是多少? [填空1]

表1-3-1 进程执行时间表

进程	到达时间	执行时间
P ₁	0.0	9
P ₂	0.4	4
P ₃	1.0	1
P ₄	5.5	4
P ₅	7	2

【参考答案】本题分步解答如下。

(1) 采用非抢占式优先级（短进程优先）调度算法，各进程执行次序如图1-3-1所示。在0时刻，进程 P_1 到达， P_1 获得处理机。由于不可抢占，因此 P_1 会一直运行到9时刻结束；在9时刻，所有进程到达，根据短进程优先调度， P_3 获得处理机，然后是 P_5 ； P_2 和 P_4 的预计运行时间一样，按照FCFS原则， P_2 优先获得处理机，最后是 P_4 。

(2) 周转时间=完成时间-作业到达时间。因此 P_1 的周转时间是 $9-0=9$ ， P_2 的周转时间是 $16-0.4=15.6$ ， P_3 的周转时间是 $10-1=9$ ， P_4 的周转时间是 $20-5.5=14.5$ ， P_5 的周转时间是 $12-7=5$ 。这5个进程的平均周转时间为 $(9+15.6+9+14.5+5)/5=10.62$ 。

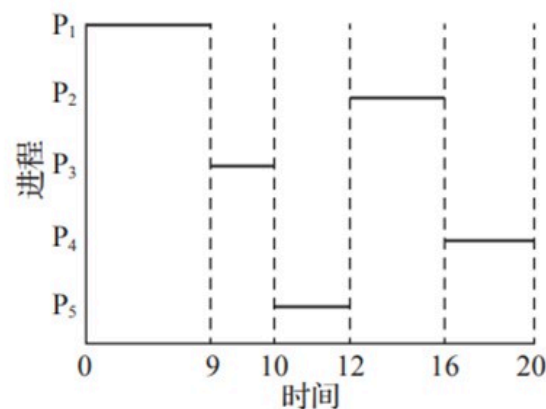


图 1-3-1 进程运行时间关系图

填空题 8分

(考研真题) 假定要在—台处理机上执行表 1-3-2 所示的作业, 且假定这些作业在时刻 0 以 1, 2, 3, 4, 5 的顺序到达。请说明分别采用 FCFS、RR (时间片为 1)、SJF 及非抢占式优先级调度算法时, 这些作业的执行情况 (优先级的高低顺序依次为 1 到 5)。

针对上述每种调度算法, 给出平均周转时间和平均带权周转时间。

FCFS 平均周转时间 [填空1], FCFS 平均带权周转时间 [填空2]。

RR 平均周转时间 [填空3], RR 平均带权周转时间 [填空4]。

SJF 平均周转时间 [填空5], SJF 平均带权周转时间 [填空6]。

优先级平均周转时间 [填空7], 优先级平均带权周转时间 [填空8]。

表 1-3-2 作业执行时间表

作业	执行时间	优先级
1	10	3
2	1	1
3	2	3
4	1	4
5	5	2

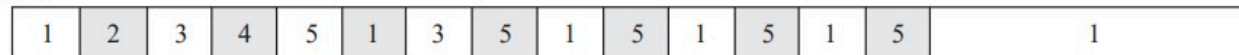
【参考答案】本题分步解答如下。

(1) 作业执行顺序如图1-3-2所示。

FCFS:



RR:



SJF:



非抢占式优先级:



图 1-3-2 作业执行顺序图

(2) 各个作业对应于各个算法的周转时间和带权周转时间见表1-3-3。

表1-3-3 作业运行情况比较表

算法	时间类型	P ₁	P ₂	P ₃	P ₄	P ₅	平均时间
	运行时间	10	1	2	1	5	3.8
FCFS	周转时间	10	11	13	14	19	13.4
	带权周转时间	1	11	6.5	14	3.8	7.26
RR	周转时间	19	2	7	4	14	9.2
	带权周转时间	1.9	2	3.5	4	2.8	2.84
SJF	周转时间	19	1	4	2	9	7
	带权周转时间	1.9	1	2	2	1.8	1.74
非抢占式 优先级	周转时间	16	1	18	19	6	12
	带权周转时间	1.6	1	9	19	1.2	6.36

因此，FCFS的平均周转时间为13.4，平均带权周转时间为7.26。RR的平均周转时间为9.2，平均带权周转时间为2.84。SJF的平均周转时间为7，平均带权周转时间为1.74。非抢占式优先级的平均周转时间为12，平均带权周转时间为6.36。注意：SJF的平均周转时间最短。

第二章（完）